

PROYECTO INTERMODULAR

PlateUp

La mejor red social de cocina multiplataforma

Módulo:	Proyecto Intermodular / 2º DAM
Autor:	Xabier López de Guereño Armada
Tutor:	Víctor Pla Soler
Centro:	IES La Vereda / La Pobla de Vallbona, València
Fecha:	25 de mayo de 2026
Repositorio:	https://github.com/XabierLDGA/PlateUp.git

I.E.S. La Vereda

La Pobla de Vallbona - València



Abstract

Castellano

PlateUp es una aplicación multiplataforma que combina el concepto de red social con el mundo de la cocina doméstica. Inspirada en plataformas como Hevy, por su sistema de registro y seguimiento de actividad, e Instagram, por su estructura social y su interfaz visual centrada en imágenes, PlateUp permite a los usuarios crear, compartir y descubrir recetas de forma dinámica, interactiva y divertida. La aplicación dispone de un feed social con filtros por categoría, un sistema de likes y follows entre usuarios, un módulo de gamificación con logros y retos, un modo guiado de cocina paso a paso con temporizadores, y una versión Android nativa generada mediante Capacitor. El backend está implementado con Java y Spring Boot 3, protegido con Spring Security y autenticación JWT, y se conecta a una base de datos MySQL alojada en Railway. El conjunto se despliega en producción mediante Railway.

English

PlateUp is a cross-platform application that combines the concept of a social network with the world of home cooking. Inspired by platforms such as Hevy, for its activity tracking system, and Instagram, for its social structure and image-centric visual interface, PlateUp allows users to create, share, and discover recipes in a dynamic, interactive and enjoyable way. The application includes a social feed with category filters, a system of likes and follows between users, a gamification module with achievements and challenges, a step-by-step guided cooking mode with timers, and a native Android version generated through Capacitor. The backend is implemented with Java and Spring Boot 3, protected with Spring Security and JWT authentication, and connects to a MySQL database hosted on Railway. The whole system is deployed in production via Railway.

Valencià

PlateUp és una aplicació multiplataforma que combina el concepte de xarxa social amb el món de la cuina domèstica. Inspirada en plataformes com Hevy, pel seu sistema de registre i seguiment d'activitat, i Instagram, per la seua estructura social i interfície visual centrada en imatges, PlateUp permet als usuaris crear, compartir i descobrir receptes de manera dinàmica, interactiva i divertida. L'aplicació disposa d'un feed social amb filtres per categoria, un sistema de likes i follows entre usuaris, un mòdul de gamificació amb assoliments i reptes, un mode guiat de cuina pas a pas amb temporitzadors, i una versió Android nativa generada amb Capacitor. El backend està implementat amb Java i Spring Boot 3, protegit amb Spring Security i autenticació JWT, i es connecta a una base de dades MySQL allotjada a Railway. El conjunt es desplega en producció mitjançant Railway.

Índice

1. Introducción.....	8
1.1 Breve descripción del proyecto.....	8
1.2 Justificación del proyecto.....	8
1.3 Objetivos del proyecto.....	8
Objetivo general.....	8
Objetivos específicos.....	9
1.4 Alcance del proyecto.....	9
1.5 Planificación temporal.....	10
2. Módulos a los que implica.....	11
2.1 Módulos de 1º DAM.....	11
2.2 Módulos de 2º DAM.....	13
3. Contexto empresarial y estudio de mercado.....	18
3.1 Creación de la empresa ficticia.....	18
3.2 Sector de actividad.....	18
3.3 Misión, visión y valores.....	18
Misión.....	18
Visión.....	18
Valores.....	19
3.4 Público objetivo.....	19
3.5 Estudio de mercado y análisis de la competencia.....	19
3.6 Análisis DAFO.....	20
3.7 Viabilidad del proyecto.....	21
Viabilidad técnica.....	21
Viabilidad económica.....	21
Viabilidad comercial.....	21
3.8 Modelo de negocio.....	21
3.9 Prevención de riesgos laborales, salud y SGE.....	21
Prevención de riesgos laborales.....	21
Salud mental y factores psicosociales.....	22
Sistemas de Gestión Empresarial.....	22
4. Análisis técnico.....	23
4.1 Arquitectura del sistema.....	23
4.2 Stack tecnológico y justificación de las elecciones.....	23
Java 21 + Spring Boot 3.5.6 (Backend).....	23
Spring Security + JWT (Autenticación).....	24
Spring Data JPA + Hibernate (Persistencia).....	24
MySQL 8 en Railway (Base de datos).....	24
Vue.js 3 + Composition API (Frontend).....	24
Tailwind CSS v4 (Estilos).....	24
Vite (Bundler).....	24
Axios (Cliente HTTP).....	25
Pinia (Gestión del estado).....	25
Capacitor (Android).....	25
Docker + Railway (Despliegue).....	25
4.3 Control de versiones.....	25
4.4 Seguridad de la aplicación.....	25
Autenticación con JWT.....	25
Filtro JWT en Spring Security.....	26

Política de acceso a endpoints.....	26
Cifrado de contraseñas.....	26
4.5 Herramientas utilizadas durante el desarrollo.....	26
Herramientas de diseño y modelado.....	26
Entornos de desarrollo integrado (IDE).....	26
Herramientas de pruebas y base de datos.....	27
Herramientas de inteligencia artificial.....	27
Control de versiones y colaboración.....	27
5. Análisis del sistema.....	28
5.1 Actores del sistema.....	28
Usuario no autenticado.....	28
Usuario autenticado.....	28
Administrador.....	28
5.2 Funcionalidades principales.....	28
Gestión de usuarios.....	28
Gestión de recetas.....	29
Interacción social.....	29
Gamificación.....	29
Modo Cocina.....	29
Panel de administración.....	29
5.3 Requisitos funcionales.....	30
5.4 Requisitos no funcionales.....	30
Rendimiento.....	30
Usabilidad.....	31
Seguridad.....	31
Disponibilidad.....	31
Mantenibilidad.....	31
Escalabilidad.....	31
Compatibilidad.....	32
5.5 Historias de usuario.....	32
Autenticación y gestión de cuenta.....	32
Gestión de recetas.....	32
Interacción social.....	33
Modo cocina y gamificación.....	33
Administración.....	34
6. Desarrollo técnico detallado.....	35
6.1 Metodología de trabajo.....	35
6.2 Desarrollo del backend (Spring Boot + API REST).....	35
Estructura del proyecto.....	35
Controladores REST implementados.....	36
Documentación automática con Springdoc OpenAPI.....	37
6.3 Desarrollo del frontend (Vue.js 3).....	37
Estructura del proyecto frontend.....	37
Interceptor de Axios.....	37
Guard de navegación en Vue Router.....	37
6.4 Aplicación móvil Android con Capacitor.....	38
6.5 Sistema de subida de imágenes.....	38
6.6 Sistema de gamificación.....	38
6.7 Pruebas realizadas.....	39
Pruebas de la API REST.....	39
Pruebas de la aplicación web.....	39
Pruebas de la aplicación Android.....	39
Pruebas con usuarios reales.....	40

6.8 Despliegue en producción (Railway).....	40
Base de datos MySQL gestionada.....	40
Backend (API REST).....	40
Frontend (Sitio estático).....	40
Docker Compose para desarrollo local.....	40
7. Base de datos.....	42
7.1 Diseño conceptual.....	42
7.2 Descripción de las entidades.....	42
Users — Usuarios.....	42
Recipes — Recetas.....	42
RecipeSteps — Pasos de receta.....	42
Ingredients — Ingredientes.....	42
RecipeIngredients — Ingredientes de receta.....	43
Utensils y RecipeUtensils — Utensilios.....	43
Follows — Seguimientos.....	43
Likes — Me gusta.....	43
Collections y CollectionRecipes — Colecciones.....	43
Achievements y UserAchievements — Logros.....	43
Challenges y UserChallenges — Retos.....	43
CookedRecipes — Recetas cocinadas.....	43
Tablas adicionales del sistema.....	44
7.3 Paso a tablas (modelo relacional completo).....	44
8. Diseño visual del proyecto (UX/UI).....	46
8.1 Filosofía de diseño y decisiones generales.....	46
8.2 AuthView — Pantalla de autenticación.....	46
8.3 HomeView — Feed principal.....	46
Encabezado.....	47
Filtros de categoría.....	47
Panel de progreso culinario (Cooking Journey).....	47
Feed de recetas.....	47
8.4 ExploreView — Exploración y búsqueda.....	47
Barra de búsqueda.....	47
Personas para descubrir (People to discover).....	47
Grid de recetas.....	48
8.5 RecipeDetailView — Detalle de receta.....	48
Hero con imagen y acciones.....	48
Metadatos de la receta.....	48
Navegación por secciones.....	48
Sección de ingredientes.....	48
Sección de pasos.....	48
8.6 CookingView — Modo cocina guiado.....	48
Temporizador global.....	49
Navegación entre pasos.....	49
Finalización de la receta.....	49
8.7 CreateView — Creación de receta.....	49
Información básica.....	49
Imagen de la receta.....	49
Metadatos de la receta.....	49
Ingredientes.....	49
Pasos de preparación.....	49
8.8 ProfileView — Perfil propio.....	50
Tarjeta de usuario.....	50
Tabs de contenido.....	50

My Recipe Book.....	50
Achievements.....	50
Opciones del menú.....	50
8.9 UserProfileView — Perfil de otros usuarios.....	50
8.10 EditProfileView — Edición del perfil.....	50
8.11 ConnectionsView — Seguidores y seguidos.....	51
8.12 AdminDashboardView — Panel de administración.....	51
9. Mejoras a largo plazo.....	52
9.1 Sistema de notificaciones en tiempo real.....	52
9.2 Recomendaciones personalizadas con IA.....	52
9.3 Integración con compra de ingredientes.....	52
9.4 Versión iOS con Capacitor.....	52
9.5 Mensajería directa entre usuarios.....	52
9.6 Mejoras en el sistema de gamificación.....	53
9.7 Internacionalización (i18n).....	53
9.8 Almacenamiento de imágenes en CDN.....	53
9.9 Sistema de reportes y moderación.....	53
9.10 Perfiles públicos y privados con sistema de solicitudes de seguimiento.....	53
10. Conclusión.....	54
11. Anexos.....	55
Anexo I. Estructura completa del repositorio.....	55
Anexo II. Endpoints completos de la API REST.....	56
Anexo III. Variables de entorno.....	58
Anexo IV. Dependencias del backend (pom.xml).....	59
Anexo V. Dependencias del frontend (package.json).....	60
Anexo VI. Configuración de Capacitor (Android).....	61
Anexo VII. Modelo entidad-relación.....	62
Anexo VIII. Identidad visual: paleta de colores y tipografía.....	63
Filosofía visual.....	63
Paleta de colores.....	63
Tipografía.....	64
Tokens de forma — border-radius.....	65
Elevación y separación.....	65
Anexo IX. Capturas de pantalla de la interfaz.....	66
AuthView (Pantalla de autenticación).....	66
HomeView (Feed principal).....	67
ExploreView (Exploración y búsqueda).....	68
RecipeDetailView (Detalle de receta).....	69
CookingView (Modo cocina guiado).....	70
CreateView (Creación de receta).....	71
ProfileView (Perfil propio).....	72
ConnectionsView (Panel de followers).....	73
AdminDashboardView (Panel de administración).....	74
Anexo X. Repositorio de GitHub.....	75
Estructura del repositorio.....	75
Historial de commits y evolución del proyecto.....	75
Anexo XI. Configuración del tiempo de expiración de tokens JWT.....	76
Configuración en application.properties.....	76
Implementación en JwtService.....	76
Anexo XII. Pruebas con Swagger UI.....	77
Organización de la colección.....	77
Casos de prueba principales.....	77
Resultados de las pruebas.....	78

1. Introducción

1.1 Breve descripción del proyecto

PlateUp es una aplicación móvil y plataforma web orientada al ámbito de la gastronomía que fusiona el concepto de red social con una plataforma completa de gestión y descubrimiento de recetas de cocina. El proyecto surge de la necesidad de crear un espacio donde los usuarios no solo puedan consultar recetas, sino también publicarlas, interactuar con otros usuarios, seguir a sus cocineros favoritos y formar parte de una comunidad activa y dinámica alrededor de la cocina.

La aplicación está desarrollada con un stack tecnológico moderno y profesional: el backend utiliza Java 21 con Spring Boot 3.5.6 y MySQL como base de datos, el frontend emplea Vue.js 3 con Tailwind CSS v4 y Vite, y la versión móvil Android se genera mediante Capacitor, que transforma la aplicación web en una aplicación instalable en dispositivos Android. El sistema completo se despliega en la nube utilizando Railway para el backend, el frontend y para la base de datos MySQL.

PlateUp no es únicamente una aplicación de recetas. Es una red social especializada donde el contenido gastronómico es el protagonista: los usuarios tienen perfil propio, pueden seguir a otras personas, recibir seguidores, explorar el contenido de la comunidad y participar en desafíos culinarios. Además, incorpora un sistema de gamificación con logros que premia la participación activa dentro de la plataforma.

1.2 Justificación del proyecto

La motivación principal de PlateUp surge de identificar un hueco claro en el mercado digital de aplicaciones gastronómicas. Actualmente existen dos tipos de plataformas bien diferenciadas: las aplicaciones de recetas tradicionales, que ofrecen contenido estático, sin interacción social y sin perfil de usuario, y las redes sociales generalistas, como Instagram o TikTok, donde el contenido de cocina convive desordenado con todo tipo de publicaciones.

PlateUp propone una solución intermedia y especializada: una plataforma donde la receta tiene una estructura formal completa (título, descripción, imagen, ingredientes con cantidades, pasos numerados, nivel de dificultad, tiempo de preparación y categoría), pero que al mismo tiempo incorpora todos los elementos sociales que hacen que los usuarios quieran volver: seguimiento de personas, feed personalizado, likes, comentarios y descubrimiento de nuevo contenido.

Desde el punto de vista técnico, el proyecto también está plenamente justificado como ejercicio académico integrador, ya que exige aplicar conocimientos de programación orientada a objetos, diseño de bases de datos relacionales, desarrollo de APIs REST, seguridad con JWT, desarrollo frontend reactivo y distribución multiplataforma, abarcando prácticamente todos los módulos cursados durante el ciclo DAM.

1.3 Objetivos del proyecto

Objetivo general

Desarrollar una aplicación completa llamada PlateUp, orientada al ámbito de la gastronomía, que combine el concepto de red social con una plataforma estructurada de recetas de cocina,

operativa tanto en entorno web como en dispositivo móvil Android, con arquitectura profesional, segura y preparada para un entorno real de producción.

Objetivos específicos

- Diseñar e implementar un sistema completo de gestión de usuarios con registro, inicio de sesión, gestión de perfil, edición de datos personales y subida de imagen de avatar.
- Desarrollar el módulo de recetas completo con todos los campos relevantes: título, descripción, imagen, tiempo de preparación, número de raciones, nivel de dificultad, categoría, lista de ingredientes con cantidades y unidades, y pasos de preparación ordenados.
- Implementar un sistema de interacción social con likes, follows, feed dinámico de recetas ordenado por fecha y sección de exploración para descubrir nuevo contenido.
- Incorporar un sistema de categorías de recetas con soporte para las categorías: Vegan, Vegetarian, Meat, Fish, Pasta, Rice, Soup, Salad, Breakfast, Dessert, Quick, Student, Healthy y Snack.
- Añadir un sistema de gamificación con logros y retos que incentive la participación activa de los usuarios.
- Desarrollar un modo cocina guiado paso a paso con temporizadores integrados por cada paso de la receta.
- Garantizar la seguridad del sistema mediante Spring Security con política stateless y autenticación mediante JSON Web Tokens.
- Conseguir que la aplicación funcione en entorno web moderno y en dispositivo Android real mediante Capacitor.
- Desplegar el sistema en producción con Railway y Railway, configurando correctamente las variables de entorno y los servicios Docker.
- Generar documentación automática de la API REST mediante Springdoc OpenAPI.

1.4 Alcance del proyecto

El proyecto PlateUp incluye el desarrollo completo de los siguientes componentes:

- Aplicación web SPA (Single Page Application) desarrollada con Vue.js 3, Vite, Tailwind CSS v4 y Axios.
- Backend REST API desarrollado con Java 21, Spring Boot 3.5.6, Spring Security y Spring Data JPA.
- Base de datos relacional MySQL 8 con un modelo de datos normalizado con más de quince tablas relacionadas.
- Sistema de autenticación stateless con JWT (JSON Web Tokens) con validez de 30 días.
- Módulo de subida de imágenes (recetas y avatares) con almacenamiento en servidor.
- Panel de administración con estadísticas y gráficos de uso de la plataforma.
- Aplicación Android generada mediante Capacitor, instalable en dispositivos reales.
- Despliegue en producción mediante Docker y Railway.

El proyecto no incluye en su versión actual: sistema de mensajería directa entre usuarios, integración con pasarela de pagos, recomendaciones basadas en inteligencia artificial, notificaciones push en tiempo real ni sistema de reportes de contenido inapropiado (aunque la tabla Reports está definida en la base de datos para uso futuro).

1.5 Planificación temporal

El desarrollo de PlateUp se organizó en fases incrementales durante aproximadamente doce semanas, siguiendo una metodología de desarrollo iterativa donde cada fase aportaba una versión funcional sobre la que continuar construyendo.

Fase	Actividades principales
Semanas 1 - 3 (Septiembre)	Análisis de requisitos, diseño del modelo entidad-relación, definición de la arquitectura del sistema y configuración del entorno de desarrollo.
Semanas 4 - 10 (Octubre - Noviembre)	Creación del proyecto Spring Boot, implementación del modelo de datos con JPA, desarrollo de repositorios y controladores REST principales.
Semanas 11 - 22 (Diciembre - Febrero)	Ampliación del backend con todos los controladores, pruebas de la API con Swagger UI y configuración de Docker y Docker Compose.
Semanas 23 - 27 (Marzo)	Desarrollo del frontend en Vue.js: estructura de vistas, conexión con la API mediante Axios, gestión del estado con Pinia y navegación con Vue Router.
Semanas 28 - 30 (Abril)	Mejora visual completa con Tailwind CSS v4, diseño responsive y pulido de la interfaz de todas las pantallas.
Semanas 31 - 33 (Abril - Mayo)	Implementación de Spring Security con JWT, cifrado BCrypt, protección de endpoints y pruebas de seguridad.
Semanas 34-36 (Mayo)	Últimos retoques funcionales, versión Android con Capacitor, despliegue en Railway y elaboración de la memoria.

2. Módulos a los que implica

El proyecto PlateUp integra de forma transversal los conocimientos adquiridos en los módulos cursados durante el ciclo formativo de Grado Superior en Desarrollo de Aplicaciones Multiplataforma. A continuación se detalla la relación de cada módulo con el proyecto, tanto de primero como de segundo curso.

2.1 Módulos de 1º DAM

Módulo	Aplicación en PlateUp
Programación	PlateUp requiere el diseño e implementación de un sistema orientado a objetos completo en Java 21. Se aplican los principios fundamentales de la POO como herencia, encapsulación, polimorfismo e interfaces en todas las capas del backend. Las entidades del sistema (User, Recipe, RecipeStep, Ingredient, Follow, Like, Achievement, Challenge, Collection, Comment, CookedRecipe) son clases Java correctamente encapsuladas con atributos privados, constructores públicos y métodos accesoros. La lógica de negocio se organiza en clases Service que implementan interfaces bien definidas, permitiendo desacoplar la presentación de la persistencia. Se utilizan estructuras de control avanzadas, colecciones genéricas (List, Optional, Set) y el manejo de excepciones personalizadas (ResourceNotFoundException, AuthenticationRequiredException) para garantizar robustez del código. Se emplean características avanzadas de Java como anotaciones (@Entity, @Service, @Repository), genéricos y el entorno Spring Boot para la inyección de dependencias.
Bases de Datos	PlateUp cuenta con un modelo relacional completo y normalizado en tercera forma normal con más de quince tablas interrelacionadas entre sí. Se aplicó el diseño cuidadoso del esquema mediante DDL con tipos de datos apropiados para cada campo (BIGINT para identificadores, VARCHAR con longitud máxima, DATETIME para timestamps, ENUM para valores predefinidos). Las claves primarias están bien definidas tanto simples como compuestas en las tablas de relaciones muchos a muchos. Las claves foráneas garantizan la integridad referencial con restricciones ON DELETE CASCADE que mantienen la coherencia de los

	datos. Se incluyeron índices en campos frecuentemente consultados y restricciones UNIQUE para campos como username y email. Los scripts de consulta incluyen SELECT complejos con múltiples JOIN entre tablas, INSERT masivo de datos de prueba, UPDATE controlado de registros y DELETE con validaciones.
Lenguajes de Marcas	JSON es el formato exclusivo de intercambio de información entre el frontend Vue.js y la API REST del backend. Cada respuesta del servidor es un documento JSON bien estructurado con propiedades descriptivas. La estructura HTML semántica de la aplicación web utiliza etiquetas apropiadas (header, nav, main, section, article) para una estructura lógica. El archivo AndroidManifest.xml del proyecto Android está correctamente estructurado con permisos, intenciones y configuraciones. Los archivos de configuración del proyecto en formato YAML (docker-compose.yml, railway.toml) y .properties permiten la parametrización del sistema sin modificar el código. Se utilizan archivos .env para la gestión de variables sensibles. La validación de los datos recibidos en la API mediante Spring Validation asegura que los datos cumplen restricciones (campos requeridos, formato de email válido, longitudes máximas).
Sistemas Informáticos	Durante el desarrollo de PlateUp se trabajó con la gestión del sistema de archivos para la subida y almacenamiento de imágenes en el servidor mediante el servicio FileStorageService. Se administraron entornos de desarrollo en sistemas Windows configurando herramientas complejas como el JDK 21, Node.js, MySQL Server, Android Studio y sus dependencias. La configuración de puertos correctos permitió que el backend escuchara en el puerto 8080, Vite en 5173 y MySQL en 3306 sin conflictos. Se gestionaron reglas de firewall local para permitir la comunicación entre componentes. Las variables de entorno del sistema se utilizaron para almacenar información sensible (claves de API, credenciales de base de datos) sin incluirlas en el código. La elaboración de esta memoria técnica documentando todas las fases del proyecto y la generación automática de documentación de la API con Swagger UI proporcionan una vista completa del sistema.
Entornos de Desarrollo	Se utilizó IntelliJ IDEA como IDE principal para el

	backend Java, aprovechando sus herramientas integradas de refactorización de código, depuración paso a paso y gestión automática de dependencias Maven. Visual Studio Code fue el editor elegido para el frontend Vue.js con extensiones específicas que proporcionan resaltado de sintaxis, autocompletado y validación para Vue y Tailwind CSS. Las pruebas de funcionamiento se realizaron sistemáticamente con Swagger UI para verificar que todos los endpoints de la API respondían correctamente con los códigos HTTP y datos esperados. La optimización del código se realizó utilizando las refactorizaciones automáticas del IDE. Los diagramas de clases del modelo de datos y los diagramas de flujo (como el flujo de autenticación JWT y el flujo de creación de receta) se elaboraron durante el diseño del sistema para documentar la arquitectura.
FOL	PlateUp incorpora la perspectiva de prevención de riesgos laborales en varios aspectos del proyecto. Se identificaron los riesgos propios del trabajo prolongado con ordenadores como la fatiga visual por exposición a pantallas, problemas posturales derivados de una posición sentada prolongada, síndrome del túnel carpiano por uso repetitivo del teclado y estrés laboral derivado de plazos ajustados. Se aplicaron medidas preventivas como pausas activas cada 45 minutos, configuración ergonómica del puesto, ajuste de brillo y temperatura de color de la pantalla. En el diseño del sistema de usuarios de PlateUp se tuvo en cuenta la protección de datos personales según la normativa GDPR y LOPD: las contraseñas nunca se almacenan en texto plano sino cifradas con BCrypt, los datos personales se limitan a los estrictamente necesarios para el funcionamiento de la plataforma, y el usuario tiene control total sobre su perfil, contenido y privacidad.

2.2 Módulos de 2º DAM

Módulo	Aplicación en PlateUp
Programación Multimedia y Dispositivos Móviles	Capacitor fue la tecnología seleccionada para el desarrollo de la versión Android de PlateUp. Se evaluaron alternativas como React Native y Flutter, eligiendo Capacitor por permitir reutilizar exactamente el

	mismo código Vue.js sin reescribirlo. El archivo <code>capacitor.config.json</code> configura el comportamiento de la aplicación en Android con parámetros como el identificador del paquete (<code>com.plateup.app</code>), el nombre de la aplicación y la URL del servidor de desarrollo. El <code>AndroidManifest.xml</code> se configuró con los permisos necesarios: <code>INTERNET</code> para la comunicación con la API, <code>READ_EXTERNAL_STORAGE</code> y <code>WRITE_EXTERNAL_STORAGE</code> para acceder a la galería de fotos. La sincronización del proyecto Vue.js compilado con el proyecto Android se realiza mediante <code>npx cap sync android</code>, que copia los archivos web al directorio <code>assets</code> del proyecto Android. La compilación del APK se realiza desde Android Studio, generando un archivo instalable en dispositivos Android reales. La prueba en dispositivo físico verificó que todas las funcionalidades funcionaban correctamente, incluyendo la navegación, la subida de imágenes desde la galería, la comunicación con la API y la persistencia de datos.
Acceso a Datos	PlateUp implementa una capa de persistencia completa mediante Spring Data JPA e Hibernate ORM. Las entidades del sistema están anotadas con <code>@Entity</code> para su mapeo automático a tablas de la base de datos. Las relaciones entre entidades se definen con <code>@OneToMany</code>, <code>@ManyToOne</code> y <code>@ManyToMany</code>, permitiendo que Hibernate genere las tablas intermedias necesarias automáticamente. Las claves primarias compuestas se implementan con <code>@EmbeddedId</code> en casos como <code>Follows</code> (<code>follower_id</code> + <code>followed_id</code>) y <code>Likes</code> (<code>user_id</code> + <code>recipe_id</code>). Los interfaces <code>Repository</code> extienden <code>JpaRepository<Entidad, TipoClave></code> y definen métodos de consulta que siguen convenciones de nombres (<code>findByUsername</code>, <code>existsByFollowerIdAndFollowedId</code>, <code>findByRecipeIdOrderByOrderIndexAsc</code>), permitiendo que Spring Data genere las consultas SQL automáticamente sin código JDBC manual. El pool de conexiones HikariCP gestiona eficientemente el número de conexiones activas hacia MySQL en Railway. El servicio <code>FileStorageService</code> encapsula toda la lógica de acceso al sistema de archivos del servidor para la gestión de imágenes subidas por los usuarios.
Programación de Servicios y Procesos	PlateUp desarrolla su componente central alrededor de la API REST. El backend implementado con Spring Boot expone 18 controladores REST que responden a

	peticiones HTTP. El protocolo HTTP se utiliza para toda la comunicación cliente-servidor, con métodos GET para lectura, POST para creación, PUT para actualización y DELETE para eliminación. El cliente Vue.js comunica con el servidor mediante Axios, configurado con un interceptor que añade automáticamente el token JWT de autenticación a todas las peticiones. Las respuestas del servidor son documentos JSON que se deserializan automáticamente a objetos JavaScript en el cliente. El rendimiento se optimiza con respuestas JSON compactas sin datos innecesarios, paginación en listados largos, y índices en la base de datos para consultas frecuentes. La disponibilidad se garantiza mediante el despliegue en Railway con Docker, permitiendo que el servicio se reinicie automáticamente si falla. La seguridad es un aspecto crítico cubierto completamente: la autenticación utiliza JWT (JSON Web Tokens) con firma HMAC-SHA256, garantizando que cada token es imposible de falsificar sin la clave secreta. Las contraseñas se cifran con BCrypt, que incluye salt aleatorio para cada hash, imposibilitando los ataques por diccionario. La política de control de acceso en Spring Security protege todos los endpoints: los de lectura son públicos, los de escritura requieren autenticación válida. La protección CORS está configurada para evitar peticiones no autorizadas desde dominios externos. La validación de entradas con @Valid en los DTOs asegura que los datos cumplen restricciones antes de procesarse. La comunicación HTTPS en producción cifra toda la información en tránsito.
Desarrollo de Interfaces	PlateUp tiene una interfaz gráfica de usuario completa desarrollada con Vue.js 3 como framework principal. Los componentes se desarrollan como Single File Components (SFC) con template, script y estilos integrados. El framework Tailwind CSS v4 proporciona todas las clases de utilidad necesarias para aplicar estilos directamente en el HTML sin escribir CSS personalizado. Se implementaron once vistas principales (HomeView, ExploreView, RecipeDetailView, CreateView, ProfileView, EditProfileView, UserProfileView, ConnectionsView, CookingView, AuthView, AdminDashboardView) que cubren todas las funcionalidades de la aplicación. Los componentes reutilizables como RecipeCard y BottomNav se utilizan en múltiples vistas para mantener consistencia visual y

	reducir duplicación de código. El diseño sigue principios de usabilidad con enfoque mobile-first: áreas de toque generosas (mínimo 44x44px), feedback inmediato mediante indicadores de carga y mensajes de error descriptivos, y navegación persistente mediante una barra inferior que facilita el acceso a las secciones principales desde cualquier punto de la aplicación. Los criterios de accesibilidad se aplicaron con contraste de colores suficiente según los estándares WCAG, textos legibles en cualquier tamaño de pantalla y estructura semántica clara. La versión web se despliega como sitio estático con HTTPS automático, y la versión Android se genera como APK mediante Capacitor empaquetando la misma aplicación web en un contenedor nativo. Las pruebas de funcionamiento se realizaron de forma sistemática en navegadores modernos y en dispositivo Android físico.
Sistemas de Gestión Empresarial (SGE)	PlateUp implementa un sistema de gestión centralizado de información basado en una base de datos relacional MySQL. La arquitectura es comparable a la de un ERP en miniatura: el núcleo central es la base de datos que almacena todos los datos de la plataforma (usuarios, recetas, interacciones, logros). El panel de administración proporciona funcionalidades de consulta y análisis similares a las que ofrece un módulo de reporting en un ERP estándar. Los KPIs mostrados (número total de usuarios, recetas publicadas, likes recibidos, comentarios activos) permiten evaluar el estado de salud de la plataforma. Los gráficos de distribución por categorías, evolución mensual de usuarios registrados y ranking de usuarios más activos facilitan la toma de decisiones basada en datos. El análisis del modelo de negocio freemium de PlateUp (publicidad para usuarios gratuitos, suscripción premium, colaboraciones con marcas) sigue los mismos principios de monetización que muchos sistemas empresariales reales.
IPE II	PlateUp se aborda en esta memoria como una propuesta empresarial completa, no solo como un proyecto técnico. Se realizó un análisis DAFO que identifica fortalezas (combinación única de red social y plataforma de recetas, diseño moderno, gamificación), debilidades (proyecto nuevo sin base de usuarios), oportunidades (crecimiento del mercado food-tech, aumento de interés en cocina casera) y amenazas

	(competencia de plataformas establecidas como Cookpad e Instagram). El estudio de mercado incluye análisis de la competencia directa e indirecta: Cookpad como referencia global de volumen de recetas, Tasty como referencia de contenido visual en vídeo, Instagram como referencia de red social generalista, Pinterest como referencia de descubrimiento visual. Se definió un modelo de negocio freemium con múltiples fuentes potenciales de ingreso: publicidad no intrusiva para usuarios gratuitos, suscripción mensual/anual para usuarios premium, colecciones de chefs profesionales, colaboraciones patrocinadas con marcas de alimentación. El análisis de viabilidad evalúa que el proyecto es técnicamente viable (todas las tecnologías son maduras y probadas), económicamente viable (costes iniciales bajos con herramientas de código abierto), y comercialmente viable (mercado objetivo identificado y diferenciación clara). La metodología Design Thinking se aplicó para validar la propuesta de valor: identificación de la necesidad (ausencia de red social especializada en gastronomía), generación de ideas (combinación de elementos de Hevy, Instagram y Cookpad), desarrollo del prototipo (implementación completa de PlateUp), y validación mediante pruebas de usuario.
--	--

3. Contexto empresarial y estudio de mercado

3.1 Creación de la empresa ficticia

Para dotar al proyecto de un enfoque más realista y profesional, se plantea la creación de una empresa ficticia denominada PlateUp Labs S.L., una startup tecnológica de nueva creación con sede en Valencia, España. PlateUp Labs nace con el objetivo de desarrollar y comercializar una plataforma digital dedicada a la gastronomía social, combinando la experiencia de usuario de las mejores redes sociales con la estructura funcional de las aplicaciones de recetas más completas del mercado.

La empresa se organiza con un equipo multidisciplinar pequeño y ágil, típico de una startup en fase inicial: un desarrollador fullstack con perfiles tanto en backend como en frontend, un diseñador UX/UI, y un responsable de producto y comunicación. La estructura organizativa es plana, lo que favorece la toma de decisiones rápida y la adaptación constante al mercado.

3.2 Sector de actividad

PlateUp Labs opera dentro del sector tecnológico, concretamente en el desarrollo de aplicaciones móviles y plataformas digitales orientadas al ámbito de la gastronomía y las redes sociales. Este sector es conocido internacionalmente como food-tech o social food, y ha experimentado un crecimiento sostenido en los últimos años, impulsado por el auge del contenido gastronómico en plataformas como YouTube, Instagram, TikTok y Pinterest.

Según datos del sector, el mercado de aplicaciones móviles relacionadas con la cocina y la gastronomía supera los 600 millones de dólares a nivel mundial, con una tasa de crecimiento anual compuesta (CAGR) superior al 8%. El segmento objetivo de PlateUp —usuarios entre 18 y 35 años con interés en cocina y activos en redes sociales— representa uno de los grupos demográficos más activos en consumo digital.

3.3 Misión, visión y valores

Misión

La misión de PlateUp Labs es facilitar una forma más social, intuitiva y motivadora de compartir recetas de cocina, conectando a personas con intereses gastronómicos comunes dentro de una misma plataforma digital. Queremos transformar la experiencia de cocinar en casa en un acto comunitario, donde cada receta sea una oportunidad de aprender, compartir e inspirarse con otros.

Visión

Nuestra visión es convertirnos en la plataforma de referencia para cocineros aficionados en el mercado hispanohablante, y posteriormente expandirnos al mercado europeo, posicionándonos como la red social especializada en gastronomía con mayor tasa de retención e interacción de usuarios. A largo plazo, PlateUp Labs aspira a integrar recomendaciones personalizadas con inteligencia artificial y a establecer alianzas estratégicas con marcas de alimentación y supermercados online.

Valores

- Comunidad: la plataforma existe para las personas y las conexiones entre ellas.
- Accesibilidad: cocinar bien no debe ser exclusivo de expertos ni de personas con recursos económicos elevados.
- Innovación: buscamos constantemente nuevas formas de mejorar la experiencia del usuario.
- Calidad: tanto en el código como en el diseño, siempre buscamos la excelencia.
- Transparencia: somos honestos con nuestros usuarios sobre el uso de sus datos y la publicidad.

3.4 Público objetivo

El público objetivo principal de PlateUp está formado por personas jóvenes y adultas con acceso a smartphone, que utilizan habitualmente redes sociales y que muestran interés por la cocina, la alimentación y el descubrimiento de nuevas recetas. Concretamente, se identifican los siguientes perfiles de usuario:

- Estudiantes universitarios (18-25 años): buscan recetas rápidas, económicas y fáciles de preparar con los ingredientes que tienen en casa. La categoría 'Student' y 'Quick' de PlateUp está pensada específicamente para ellos.
- Personas que viven solas o en piso compartido (22-32 años): necesitan organizar sus comidas semanales y buscan inspiración para no repetir siempre lo mismo. Valoran la estructura clara de ingredientes y pasos.
- Aficionados a la cocina saludable (25-40 años): interesados en las categorías Vegan, Vegetarian y Healthy. Siguen a influencers de alimentación y comparten sus propias recetas.
- Cocineros amateur creativos (20-35 años): disfrutan creando recetas propias y quieren compartirlas con una comunidad que las valore. Son los creadores de contenido más activos de la plataforma.
- Familias que buscan nuevas ideas para el menú semanal (30-45 años): utilizan la plataforma principalmente para descubrir recetas y seguir a cocineros de confianza.

3.5 Estudio de mercado y análisis de la competencia

El análisis competitivo de PlateUp identifica los siguientes tipos de competidores directos e indirectos:

Competidor	Análisis
Cookpad	La plataforma de recetas más grande del mundo. Punto fuerte: gran volumen de recetas. Debilidad: interfaz anticuada, poca interacción social real.
Tasty (BuzzFeed)	Recetas en formato vídeo muy viral. Punto fuerte: contenido visual atractivo. Debilidad: no hay perfiles de usuario reales ni sistema de follows.
Instagram / TikTok	Redes sociales generalistas con mucho contenido de cocina. Punto fuerte: alcance masivo. Debilidad: no existe estructura de receta; el contenido es efímero y desorganizado.

Pinterest	Plataforma de descubrimiento visual. Punto fuerte: inspiración visual. Debilidad: redirige a blogs externos, sin comunidad propia ni interacción directa.
AllRecipes	Plataforma de recetas con comunidad. Punto fuerte: valoraciones y comentarios. Debilidad: diseño web orientado a desktop, sin experiencia móvil fluida.

PlateUp se diferencia de todos estos competidores por ofrecer simultáneamente: estructura completa de receta con ingredientes y pasos detallados, sistema social completo con follows y likes, perfiles de usuario personalizados con estadísticas, categorías bien organizadas, sistema de gamificación y modo cocina guiado, todo dentro de una misma aplicación con diseño mobile-first.

3.6 Análisis DAFO

Cuadrante	Contenido
Fortalezas	Combinación única de red social y plataforma de recetas en una sola herramienta. Diseño moderno, limpio y centrado en experiencia móvil. Sistema de gamificación que incentiva la participación. Arquitectura técnica sólida y escalable. Distribución multiplataforma (web + Android). Modo cocina guiado paso a paso con temporizadores.
Debilidades	Proyecto de nueva creación sin reconocimiento de marca ni base de usuarios establecida. Recursos económicos y humanos limitados propios de una startup inicial. Necesidad de masa crítica de usuarios para que la parte social sea atractiva. Sin versión iOS en la versión actual. Almacenamiento de imágenes en servidor local (no CDN).
Oportunidades	Crecimiento sostenido del mercado food-tech a nivel global. Aumento del interés por la cocina casera y la alimentación saludable tras la pandemia. Escasez de plataformas especializadas con buena experiencia móvil en español. Posibilidad de integración con IA para recomendaciones personalizadas. Alianzas potenciales con marcas de alimentación y supermercados online.
Amenazas	Gran presencia de plataformas ya establecidas con millones de usuarios. Las redes sociales generalistas como TikTok e Instagram copan gran parte de la atención gastronómica. Cambios rápidos en las tendencias de consumo digital. Riesgo de que competidores grandes copien las funcionalidades diferenciadas de PlateUp.

3.7 Viabilidad del proyecto

Viabilidad técnica

El proyecto es técnicamente viable al cien por cien. Todas las tecnologías empleadas son maduras, ampliamente documentadas, con comunidades activas y soporte a largo plazo. Java y Spring Boot son el estándar de facto en desarrollo backend empresarial. Vue.js es uno de los frameworks frontend más adoptados a nivel mundial. MySQL es una de las bases de datos relacionales más utilizadas. Capacitor está mantenido por Ionic y cuenta con soporte activo para Android e iOS. El stack tecnológico elegido es coherente, escalable y perfectamente adecuado para el tipo de aplicación desarrollada.

Viabilidad económica

El coste de desarrollo inicial es muy bajo, dado que todas las herramientas y tecnologías utilizadas son gratuitas y de código abierto. El despliegue en Railway en el plan gratuito permite tener el backend y el frontend accesibles online sin coste mensual durante la fase de validación. Railway ofrece un plan gratuito para MySQL que es suficiente para las pruebas iniciales. A medida que la plataforma escale, los costes de infraestructura crecerán de forma proporcional al uso, lo cual es el modelo habitual en aplicaciones SaaS modernas.

Viabilidad comercial

El mercado objetivo está claramente identificado y tiene un volumen significativo. La propuesta de valor de PlateUp es diferenciada respecto a los competidores existentes. El modelo de negocio freemium es viable desde el primer día y permite crecer sin barreras de entrada para los usuarios. La monetización puede activarse gradualmente a medida que la base de usuarios crezca.

3.8 Modelo de negocio

PlateUp adopta un modelo de negocio freemium con las siguientes fuentes de ingresos potenciales:

- Publicidad no intrusiva: anuncios contextuales relacionados con ingredientes, utensilios de cocina y productos alimentarios para usuarios del plan gratuito.
- Suscripción Premium mensual/anual: acceso sin publicidad, colecciones privadas ilimitadas, estadísticas avanzadas del perfil, insignias exclusivas y acceso anticipado a nuevas funcionalidades.
- Recetas y colecciones de chefs reconocidos: contenido premium de cocineros profesionales disponible por suscripción.
- Colaboraciones con marcas de alimentación: recetas patrocinadas por marcas del sector, perfectamente etiquetadas como contenido patrocinado.
- Integración con supermercados online: comisión por la compra de ingredientes directamente desde la lista de una receta.

3.9 Prevención de riesgos laborales, salud y SGE

Prevención de riesgos laborales

Durante el desarrollo de PlateUp se realizó un análisis de los riesgos laborales asociados al trabajo de programación. Los principales riesgos identificados fueron: fatiga visual por exposición prolongada a pantallas, problemas posturales derivados de una posición sentada

incorrecta, síndrome del túnel carpiano por uso excesivo del teclado y ratón, y fatiga mental derivada de la resolución de problemas complejos durante muchas horas seguidas.

Las medidas preventivas aplicadas fueron: uso de pantalla con configuración de brillo y temperatura de color adaptada, paradas activas cada 45-60 minutos de trabajo continuo, configuración ergonómica del puesto de trabajo, y uso de técnicas de gestión del tiempo como Pomodoro para alternar trabajo y descanso.

En cuanto a los riesgos de seguridad informática inherentes a la propia aplicación, se implementaron las siguientes medidas: cifrado de contraseñas con BCrypt, autenticación JWT con expiración, validación de datos en el backend, control de acceso en todos los endpoints protegidos, configuración CORS restrictiva y almacenamiento seguro de variables sensibles como claves JWT y credenciales de base de datos mediante variables de entorno.

Salud mental y factores psicosociales

El desarrollo de software conlleva riesgos psicosociales que no siempre se tienen en cuenta. Entre los más relevantes identificados durante el desarrollo de PlateUp destacan el estrés derivado de los plazos de entrega, la frustración ante errores difíciles de depurar, la sensación de bloqueo cuando una funcionalidad no avanza como se esperaba, y la presión de integrar tantas tecnologías diferentes a la vez.

Como estrategias de gestión se aplicaron: división del proyecto en tareas pequeñas y manejables con commits frecuentes que aportaban sensación de progreso constante, uso de una lista de tareas pendientes priorizada, comunicación regular con el tutor para orientación técnica, y aceptación de que no todos los problemas tienen solución inmediata.

Sistemas de Gestión Empresarial

Desde la perspectiva de los Sistemas de Gestión Empresarial, PlateUp puede entenderse como un sistema de información centralizado que gestiona datos de usuarios, contenidos, interacciones y estadísticas. La base de datos relacional MySQL actúa como el núcleo de información del sistema, análogo al módulo central de un ERP. El panel de administración de PlateUp ofrece funcionalidades comparables a los módulos de reporting y analytics de los sistemas ERP modernos, permitiendo visualizar métricas clave como el número de usuarios registrados, recetas publicadas, likes recibidos y crecimiento mensual de la plataforma.

4. Análisis técnico

4.1 Arquitectura del sistema

PlateUp sigue una arquitectura de tres capas bien diferenciadas (cliente, servidor y base de datos), con separación total entre frontend y backend. La comunicación entre las capas se realiza exclusivamente mediante una API REST que devuelve y recibe datos en formato JSON. Esta arquitectura se conoce como SPA + REST API y es una de las más adoptadas en el desarrollo de aplicaciones web y móviles modernas.

Componente	Función
Cliente web (Vue.js SPA)	Aplicación de página única que se ejecuta en el navegador del usuario. Gestiona la interfaz, la navegación y el estado. Se comunica con el backend exclusivamente mediante llamadas HTTP asíncronas con Axios.
Cliente Android (Capacitor)	Wrapper nativo que empaqueta la aplicación Vue.js como una app Android. Permite acceder a APIs nativas del dispositivo como cámara y almacenamiento.
Backend REST (Spring Boot)	Servidor Java que expone la API REST. Gestiona la autenticación, la lógica de negocio y el acceso a datos. Organizado en capas: Controller → Service → Repository → Entity.
Base de datos (MySQL en Railway)	Base de datos relacional gestionada en la nube mediante Railway. Almacena toda la información persistente del sistema con alta disponibilidad.
Almacenamiento de imágenes	Sistema de subida de archivos con almacenamiento en el servidor de Railway. Las URLs de las imágenes se guardan en la base de datos para ser recuperadas por el frontend.

4.2 Stack tecnológico y justificación de las elecciones

Cada tecnología del stack de PlateUp fue elegida por razones concretas, evaluando alternativas y seleccionando la opción más adecuada para los requisitos del proyecto.

Java 21 + Spring Boot 3.5.6 (Backend)

Java fue elegido como lenguaje de backend por ser el lenguaje principal del ciclo DAM y por su solidez en el desarrollo de aplicaciones empresariales. La versión 21 aporta características modernas como records y pattern matching que mejoran la legibilidad del código. Spring Boot fue la elección natural sobre alternativas como Micronaut o Quarkus por ser el framework Java más utilizado en el mercado, con la mayor comunidad de soporte, la documentación más completa y la mejor integración con Spring Security y Spring Data JPA. La versión 3.x aporta soporte para Jakarta EE 10 y Java 17+, con mejoras significativas en rendimiento respecto a la versión 2.

Spring Security + JWT (Autenticación)

Spring Security es el estándar de seguridad para aplicaciones Spring. Se eligió sobre alternativas como Auth0 o Firebase Authentication por permitir un control total sobre el proceso de autenticación sin dependencias de servicios externos. La autenticación con JWT (JSON Web Tokens) se eligió sobre las sesiones de servidor porque es stateless: cada petición incluye toda la información necesaria para autenticar al usuario en el propio token, lo que hace el sistema horizontalmente escalable sin necesidad de sesiones compartidas entre instancias. Se utilizó la librería JJWT 0.12.5, la implementación de JWT para Java más madura y ampliamente utilizada.

Spring Data JPA + Hibernate (Persistencia)

Spring Data JPA se eligió como capa de persistencia por su integración perfecta con Spring Boot y por eliminar la necesidad de escribir código JDBC manual. Hibernate actúa como proveedor JPA, gestionando automáticamente el mapeo objeto-relacional (ORM). Esta elección permitió desarrollar toda la capa de acceso a datos de PlateUp declarativamente, mediante interfaces Repository con métodos derivados del nombre del método (findByUsername, findByRecipeld, etc.), reduciendo drásticamente el código repetitivo.

MySQL 8 en Railway (Base de datos)

MySQL fue elegido sobre alternativas como PostgreSQL o H2 por ser la base de datos relacional más utilizada en el mercado y por la familiarización adquirida durante el módulo de Bases de Datos del ciclo. La versión 8 aporta mejoras importantes en rendimiento, soporte completo para el estándar SQL y mejor gestión del charset UTF-8 (utf8mb4) necesario para soporte de emojis y caracteres especiales en los contenidos de la plataforma. Railway fue elegido como proveedor de base de datos gestionada en producción por ofrecer un plan gratuito con alta disponibilidad, copias de seguridad automáticas y conexión SSL, eliminando la necesidad de gestionar el servidor de base de datos manualmente.

Vue.js 3 + Composition API (Frontend)

Vue.js 3 fue elegido sobre alternativas como React o Angular por su curva de aprendizaje más suave, su sintaxis clara y su excelente documentación en español. La Composition API, introducida en Vue 3, permite organizar el código por funcionalidades en lugar de por opciones (data, methods, computed), lo que mejora la reutilización de lógica entre componentes. Frente a React, Vue tiene la ventaja de que los componentes SFC (Single File Components) agrupan template, script y estilos en un mismo archivo, lo que facilita la organización del proyecto.

Tailwind CSS v4 (Estilos)

Tailwind CSS fue elegido sobre alternativas como Bootstrap, Bulma o CSS puro por su enfoque utility-first que permite construir interfaces complejas directamente en el HTML sin salir del archivo del componente. La versión 4, que es la más reciente y utilizada en el proyecto, mejora significativamente el rendimiento de compilación respecto a la versión 3 mediante el uso del motor Lightning CSS. Frente a Bootstrap, Tailwind ofrece mucha más flexibilidad para crear diseños personalizados sin necesidad de sobrescribir estilos predefinidos.

Vite (Bundler)

Vite fue elegido sobre Webpack por su velocidad de arranque en modo desarrollo y su configuración mucho más sencilla. Vite utiliza módulos ES nativos del navegador durante el desarrollo, lo que elimina el paso de bundling y reduce drásticamente el tiempo de recarga al

modificar el código. En producción genera un bundle optimizado con Rollup. Es la herramienta recomendada por el equipo de Vue.js para nuevos proyectos.

Axios (Cliente HTTP)

Axios fue elegido sobre la API Fetch nativa del navegador por su mejor manejo de errores, su soporte para interceptores HTTP (que en PlateUp se utilizan para añadir automáticamente el token JWT a todas las peticiones) y su compatibilidad con el entorno de Capacitor en Android.

Pinia (Gestión del estado)

Pinia es la librería de gestión del estado oficial recomendada por el equipo de Vue.js para Vue 3, sustituyendo a Vuex. Se eligió por su API más simple e intuitiva, su integración nativa con Vue DevTools y su soporte completo para TypeScript. En PlateUp, Pinia gestiona el estado de autenticación del usuario mediante authStore.js.

Capacitor (Android)

Capacitor fue elegido sobre React Native o Flutter por permitir reutilizar exactamente el mismo código de la aplicación web (Vue.js + HTML + CSS) sin necesidad de reescribirlo. Esto reduce drásticamente el tiempo de desarrollo de la versión móvil. Capacitor envuelve la aplicación web en un WebView nativo del sistema operativo y proporciona acceso a las APIs nativas del dispositivo mediante plugins. Es mantenido por Ionic, una empresa con sólida trayectoria en el desarrollo de aplicaciones multiplataforma.

Docker + Railway (Despliegue)

Docker fue elegido para el despliegue por garantizar que la aplicación funciona exactamente igual en cualquier entorno, eliminando el clásico problema de 'funciona en mi máquina'. Railway fue elegido como plataforma de despliegue en la nube por ofrecer un plan gratuito que permite desplegar tanto servicios web Docker como sitios estáticos, con HTTPS automático y despliegue continuo desde GitHub. Railway fue elegido para la base de datos gestionada por las razones mencionadas anteriormente.

4.3 Control de versiones

Durante todo el desarrollo de PlateUp se utilizó Git como sistema de control de versiones distribuido, con el repositorio remoto alojado en GitHub. El flujo de trabajo siguió un modelo de commits frecuentes y descriptivos en la rama principal (master), lo que permite seguir la evolución del proyecto commit a commit y revertir cambios si fuera necesario.

El historial completo de commits en Git proporciona trazabilidad total del desarrollo, permitiendo ver exactamente qué cambios se hicieron en cada fase del proyecto. El repositorio está disponible en: <https://github.com/XabierLDGA/PlateUp.git>

4.4 Seguridad de la aplicación

La seguridad de PlateUp se implementa en múltiples capas, garantizando que solo los usuarios autorizados puedan realizar acciones que requieren autenticación.

Autenticación con JWT

Al registrarse o iniciar sesión correctamente, el servidor genera un JWT firmado con una clave secreta HMAC-SHA256 configurada en application.properties. El token tiene una validez de

30 días (2.592.000.000 milisegundos) y contiene como claims el username, el userId y el email del usuario.

Filtro JWT en Spring Security

La clase `JwtAuthenticationFilter` extiende `OncePerRequestFilter` e intercepta todas las peticiones entrantes. Extrae el token de la cabecera `Authorization`, lo valida usando `JwtService` (que verifica la firma y la expiración), obtiene el username del subject del token, carga el usuario de la base de datos y establece el contexto de seguridad de Spring Security para que los controladores puedan identificar al usuario autenticado mediante `@AuthenticationPrincipal`.

Política de acceso a endpoints

La clase `SecurityConfig` configura la política de acceso con los siguientes criterios: los endpoints `/api/users/login` y `/api/users/register` son públicos. Los endpoints GET de `/api/**` son públicos (cualquiera puede leer recetas). Los endpoints de escritura (POST, PUT, DELETE) requieren autenticación. Los endpoints de `/api/admin/**` requieren autenticación. Las peticiones OPTIONS se permiten sin restricción para el correcto funcionamiento del CORS preflight.

Cifrado de contraseñas

Las contraseñas de los usuarios se cifran con `BCrypt` antes de almacenarse en la base de datos. `BCrypt` incorpora un salt aleatorio en cada hash, lo que garantiza que dos contraseñas idénticas generan hashes distintos y hace prácticamente imposible los ataques de diccionario o rainbow table. El bean `PasswordEncoder` de Spring Security se configura con `BCryptPasswordEncoder`.

4.5 Herramientas utilizadas durante el desarrollo

A lo largo del desarrollo de PlateUp se utilizaron diversas herramientas de apoyo que, sin formar parte del stack tecnológico principal, resultaron fundamentales para el diseño, la documentación, las pruebas y la productividad general del proyecto. A continuación se detallan las más relevantes.

Herramientas de diseño y modelado

`draw.io` (`diagrams.net`) fue la herramienta utilizada para elaborar el diagrama entidad-relación de la base de datos y los diagramas de arquitectura del sistema. Su interfaz web gratuita permite crear diagramas técnicos de calidad sin instalación, con soporte para formas UML, ERD y diagramas de flujo. Los archivos se guardan en formato XML y pueden exportarse en PNG o PDF para incluirse en la documentación.

Figma se utilizó en la fase inicial de diseño para crear prototipos de baja y media fidelidad de las pantallas principales de PlateUp, permitiendo visualizar la estructura de la interfaz antes de implementarla en código. Su modo de colaboración en tiempo real y su sistema de componentes reutilizables facilitaron la definición de la paleta de colores, la tipografía y los patrones de navegación de la aplicación.

Entornos de desarrollo integrado (IDE)

IntelliJ IDEA fue el IDE principal para el desarrollo del backend en Java con Spring Boot. Sus herramientas integradas de refactorización, depuración paso a paso, soporte nativo para anotaciones Spring y gestión automática de dependencias Maven lo convierten en la opción más adecuada para proyectos Java empresariales de esta envergadura.

Visual Studio Code fue el editor utilizado para el desarrollo del frontend en Vue.js 3. Con extensiones como Volar (soporte oficial para Vue 3), Tailwind CSS IntelliSense y ESLint, proporcionó autocompletado, resaltado de sintaxis y validación de código en tiempo real. Es el editor recomendado por el equipo de Vue.js para proyectos frontend modernos.

Android Studio fue necesario para la compilación del APK de la versión Android generada con Capacitor, así como para ejecutar el emulador durante las pruebas de la aplicación móvil.

Herramientas de pruebas y base de datos

Swagger UI fue la herramienta utilizada para probar todos los endpoints de la API REST durante el desarrollo. Permite enviar peticiones HTTP con cualquier método (GET, POST, PUT, DELETE), configurar cabeceras de autorización con el token JWT y definir cuerpos de petición en JSON directamente desde el navegador. Las pruebas sistemáticas con Swagger UI permitieron verificar que todos los controladores respondían con los códigos HTTP y los cuerpos de respuesta correctos.

Mermaid AI fue la herramienta utilizada para elaborar los diagramas del proyecto, como el diagrama entidad-relación y los diagramas de arquitectura del sistema. Su interfaz web permite definir diagramas mediante sintaxis de texto y exportarlos en formato imagen para incluirlos en la documentación.

Docker Desktop permitió levantar el entorno de desarrollo completo (backend, frontend y MySQL) mediante un único comando `docker-compose up`, garantizando que todos los servicios arrancasen correctamente con la configuración definida en el archivo `docker-compose.yml`.

Herramientas de inteligencia artificial

ChatGPT Pro (OpenAI) se utilizó como asistente de consulta técnica durante el desarrollo, especialmente útil para resolver dudas concretas sobre configuraciones de Spring Security, comportamientos específicos de JPA/Hibernate, y patrones de diseño en Vue.js 3. También se empleó para obtener explicaciones detalladas de errores de compilación o excepciones de Spring Boot que no quedaban claras con la documentación oficial.

Claude Code (Anthropic) fue la herramienta de IA utilizada directamente en el terminal durante el desarrollo del backend. Resultó especialmente útil para refactorizaciones complejas, generación de DTOs y resoluciones con errores complejos.

Control de versiones y colaboración

Git fue el sistema de control de versiones utilizado durante todo el desarrollo, con commits frecuentes y mensajes descriptivos. GitHub aloja el repositorio remoto, actúa como copia de seguridad del proyecto y se integra con Railway para el despliegue continuo automático: cada push a la rama master desencadena automáticamente un nuevo despliegue del backend y el frontend en Railway.

5. Análisis del sistema

5.1 Actores del sistema

Usuario no autenticado

Es el usuario que accede a la aplicación sin haber iniciado sesión. Su acceso está limitado exclusivamente a las pantallas de registro e inicio de sesión (AuthView). Puede crear una cuenta nueva proporcionando username, email y contraseña, o iniciar sesión con sus credenciales si ya está registrado. El router de Vue.js implementa una guarda de navegación que redirige automáticamente al login cualquier intento de acceder a rutas protegidas sin autenticación.

Usuario autenticado

Es el usuario principal de la aplicación. Tras iniciar sesión y recibir el token JWT, tiene acceso completo a todas las funcionalidades de PlateUp: ver el feed, crear recetas, interactuar con otros usuarios, gestionar su perfil y acceder al modo cocina. El sistema identifica al usuario autenticado en cada petición al backend mediante el token JWT incluido en la cabecera de autorización.

Administrador

Es un usuario con rol de administrador que tiene acceso adicional al panel de administración (/admin). El router implementa una guarda adminOnly que verifica que el usuario tiene permisos de administrador antes de permitir el acceso a esta ruta. El panel de administración muestra estadísticas globales de la plataforma: número total de usuarios, recetas, likes, así como gráficos de distribución por categorías y evolución mensual.

5.2 Funcionalidades principales

A continuación se detallan todas las funcionalidades implementadas en PlateUp, organizadas por módulo funcional:

Gestión de usuarios

- Registro de nuevos usuarios con validación de datos (username único, email único, contraseña mínima).
- Inicio de sesión con credenciales (username/email + contraseña) y generación de token JWT.
- Gestión del perfil personal: nombre para mostrar, biografía y URL de avatar.
- Subida de imagen de avatar en formatos PNG, JPEG y WebP.
- Edición de datos del perfil desde la vista EditProfileView.
- Visualización del perfil propio con estadísticas: recetas publicadas, seguidores y seguidos.
- Visualización del perfil de otros usuarios con opción de seguir/dejar de seguir.
- Cierre de sesión con limpieza del store de autenticación.

Gestión de recetas

- Creación de recetas con todos los campos: título, descripción, imagen, categoría, tiempo, dificultad y raciones.
- Adición de ingredientes con nombre, cantidad decimal y unidad de medida.
- Adición de pasos de preparación numerados con texto y tiempo de temporizador opcional.
- Subida de imagen principal de la receta en formatos PNG, JPEG y WebP.
- Visualización del detalle completo de cualquier receta con todas sus secciones.
- Eliminación de recetas propias desde el perfil del usuario.
- Sistema de categorías con 14 opciones: Vegan, Vegetarian, Meat, Fish, Pasta, Rice, Soup, Salad, Breakfast, Dessert, Quick, Student, Healthy y Snack.

Interacción social

- Sistema de likes: dar like y quitar like a cualquier receta con contador visible.
- Sistema de follows: seguir y dejar de seguir a cualquier usuario.
- Feed dinámico en la pantalla principal con todas las recetas ordenadas por fecha.
- Filtrado del feed por categoría de receta.
- Pantalla Explore para descubrir usuarios y recetas de toda la comunidad.
- Búsqueda en tiempo real de recetas y usuarios por nombre.
- Lista de seguidores y seguidos accesible desde el perfil.

Gamificación

- Sistema de logros (Achievements) con nombre, descripción, icono, categoría y puntos asociados.
- Asignación de logros a usuarios (UserAchievements) con registro de la fecha de obtención y nivel.
- Sistema de retos (Challenges) con fecha de inicio, fin, objetivo y puntos de recompensa.
- Seguimiento del progreso de cada usuario en cada reto activo (UserChallenges).
- Visualización de logros obtenidos en la pestaña Achievements del perfil.
- Panel de progreso culinario en la pantalla Home con racha de días, recetas cocinadas y puntos totales.

Modo Cocina

- Acceso al modo cocina desde el botón Start Cooking en el detalle de la receta.
- Presentación guiada paso a paso de los pasos de preparación.
- Temporizador global con inicio, pausa y reanudación para el tiempo total de cocina.
- Temporizador individual por paso cuando el paso tiene timer_seconds definido.
- Barra de progreso visual que indica el avance en los pasos de la receta.
- Posibilidad de guardar la receta como cocinada con el tiempo transcurrido (CookedRecipes).

Panel de administración

- KPIs globales: número total de usuarios, recetas, likes y comentarios.

- Gráfico de donut con distribución de recetas por categoría.
- Gráfico de barras con evolución de nuevos usuarios en los últimos 6 meses.
- Gráfico de barras con ranking de usuarios por número de recetas publicadas.

5.3 Requisitos funcionales

Los requisitos funcionales definen qué debe hacer el sistema para satisfacer las necesidades de los usuarios:

- RF-01: El sistema debe permitir el registro de nuevos usuarios con username único, email único y contraseña. El sistema debe validar que no existan usuarios con el mismo username o email antes de crear la cuenta.
- RF-02: El sistema debe permitir el inicio de sesión con username o email y contraseña. Tras una autenticación correcta, el sistema debe devolver un token JWT válido durante 30 días.
- RF-03: El sistema debe proteger todos los endpoints de escritura (POST, PUT, DELETE) exigiendo un token JWT válido en la cabecera Authorization de la petición.
- RF-04: El sistema debe permitir a los usuarios autenticados crear recetas con título, descripción, imagen, categoría, tiempo, dificultad, raciones, lista de ingredientes y pasos de preparación.
- RF-05: El sistema debe mostrar un feed de recetas en la pantalla principal ordenado por fecha de publicación, con posibilidad de filtrar por categoría.
- RF-06: El sistema debe permitir dar y quitar like a cualquier receta. Cada usuario puede dar un único like por receta.
- RF-07: El sistema debe permitir seguir y dejar de seguir a cualquier otro usuario. El sistema debe registrar la fecha de inicio del seguimiento.
- RF-08: El sistema debe permitir la búsqueda de recetas y usuarios por nombre en tiempo real.
- RF-09: El sistema debe mostrar el perfil de cada usuario con sus recetas publicadas, número de seguidores, número de seguidos y logros obtenidos.
- RF-10: El sistema debe proporcionar un modo cocina guiado con navegación paso a paso y temporizadores.
- RF-11: El sistema debe permitir subir imágenes de recetas y de perfil en formatos PNG, JPEG y WebP.
- RF-12: El sistema debe proporcionar un panel de administración con estadísticas globales de la plataforma, accesible únicamente para usuarios con rol de administrador.

5.4 Requisitos no funcionales

Rendimiento

La API REST del backend debe responder a las peticiones de lectura (GET) en menos de 500 milisegundos en condiciones normales de carga. Las operaciones de escritura (POST, PUT, DELETE) pueden tolerar hasta 1 segundo de respuesta. El frontend debe cargar el feed inicial en menos de 2 segundos en una conexión de 4G estándar. Las imágenes deben optimizarse antes de enviarse al servidor para reducir el tiempo de carga.

El sistema de pool de conexiones de HikariCP está configurado con un timeout de conexión de 30 segundos y un tiempo de inicialización de 60 segundos, parámetros adecuados para el entorno de despliegue en Railway.

Usabilidad

La interfaz de PlateUp debe ser intuitiva y no requerir formación previa para su uso. Las acciones principales (crear receta, dar like, seguir usuario) deben ser accesibles en un máximo de dos toques desde cualquier pantalla. La navegación principal mediante la barra inferior (BottomNav) debe estar siempre visible en todas las pantallas principales. El diseño debe ser responsive y funcionar correctamente tanto en pantallas de 360px (dispositivos Android pequeños) como en pantallas de escritorio de 1440px.

Los formularios deben proporcionar feedback inmediato al usuario: mensajes de error descriptivos cuando los datos son incorrectos, indicadores de carga durante las operaciones asíncronas, y confirmación visual de las acciones completadas con éxito.

Seguridad

El sistema debe garantizar que ningún usuario pueda modificar ni eliminar contenido que no le pertenece. Los tokens JWT deben estar firmados con una clave secreta de al menos 256 bits para prevenir ataques de falsificación. Las contraseñas deben almacenarse siempre cifradas con BCrypt, nunca en texto plano. La comunicación entre el cliente y el servidor debe realizarse siempre mediante HTTPS en producción. Las variables sensibles (clave JWT, credenciales de base de datos) no deben incluirse en el código fuente del repositorio.

El sistema debe implementar protección CORS correctamente configurada para evitar peticiones no autorizadas desde dominios externos. Los archivos subidos al servidor deben validarse en tipo y tamaño antes de almacenarse para prevenir la subida de archivos maliciosos.

Disponibilidad

El sistema debe estar disponible al menos el 99% del tiempo durante el período de evaluación del proyecto. El despliegue en Railway con plan gratuito introduce la limitación de un límite mensual de horas de ejecución, por lo que el servidor puede dejar de responder si se agotan los créditos del mes, lo cual es aceptable en el contexto de un proyecto académico. Para un entorno de producción real, se optaría por un plan de pago que garantice disponibilidad continua sin restricción de horas.

Mantenibilidad

El código fuente del proyecto debe seguir los principios SOLID y estar organizado en capas bien diferenciadas (presentación, lógica de negocio, acceso a datos) para facilitar su mantenimiento y extensión futura. El backend debe estar documentado mediante Springdoc OpenAPI, accesible en /swagger-ui/index.html. El código Vue.js debe seguir las convenciones del estilo oficial de Vue.js con componentes SFC bien organizados.

Escalabilidad

La arquitectura stateless del backend (sin sesiones de servidor, con autenticación JWT) permite escalar horizontalmente añadiendo nuevas instancias del servicio sin necesidad de compartir estado entre ellas. La separación entre frontend y backend permite escalar cada componente de forma independiente según las necesidades de carga. La base de datos MySQL en Railway puede aumentarse a planes superiores conforme crezca el volumen de datos y consultas.

Compatibilidad

La aplicación web debe funcionar correctamente en los navegadores modernos más utilizados: Chrome 90+, Firefox 88+, Safari 14+ y Edge 90+. La aplicación Android debe funcionar en dispositivos con Android 7.0 (API level 24) o superior. La interfaz debe adaptarse correctamente a los tamaños de pantalla más comunes: 360px, 414px, 768px y 1440px de ancho.

5.5 Historias de usuario

Las historias de usuario describen las funcionalidades del sistema desde el punto de vista del usuario final, siguiendo el formato estándar: Como [rol], quiero [acción], para [beneficio]. Cada historia incluye sus criterios de aceptación que definen cuándo se considera completada.

Autenticación y gestión de cuenta

HU-01 — REGISTRO DE USUARIO

Como usuario nuevo, quiero registrarme con un nombre de usuario, email y contraseña, para poder acceder a todas las funcionalidades de PlateUp.

Criterios de aceptación: el sistema valida que el username y el email no estén ya registrados; los tres campos son obligatorios y no pueden estar vacíos; si el username ya existe devuelve el mensaje "That username is already in use" y si el email ya existe devuelve "That email is already in use"; tras el registro el usuario queda autenticado directamente y se redirige a su perfil.

HU-02 — INICIO DE SESIÓN

Como usuario registrado, quiero iniciar sesión con mi username o email y contraseña, para acceder a mi cuenta y mi contenido.

Criterios de aceptación: el sistema acepta tanto username como email como identificador; devuelve un token JWT válido durante 30 días si las credenciales son correctas; si son incorrectas muestra el mensaje genérico "The username/email or password is incorrect" sin indicar cuál de los dos campos falla; el token se almacena automáticamente y se incluye en todas las peticiones posteriores.

HU-03 — EDICIÓN DE PERFIL

Como usuario autenticado, quiero editar mi foto de perfil, para personalizar mi presencia en la comunidad.

Criterios de aceptación: los cambios se guardan en la base de datos al confirmar; la imagen de avatar se previsualiza localmente antes de subirla al servidor; solo el propio usuario puede modificar su cuenta (el backend devuelve 403 si otro intenta hacerlo).

Gestión de recetas

HU-04 — PUBLICAR UNA RECETA

Como usuario autenticado, quiero publicar una receta con título, descripción, imagen, ingredientes y pasos de preparación, para compartirla con la comunidad.

Criterios de aceptación: el formulario requiere como mínimo título, descripción, categoría, tiempo y número de raciones; los ingredientes y pasos vacíos se ignoran automáticamente al publicar; la imagen se previsualiza antes de publicar; si no se sube imagen la receta se publica sin ella.

HU-05 — CONSULTAR DETALLE DE UNA RECETA

Como usuario, quiero ver todos los detalles de una receta (ingredientes, pasos, tiempo, dificultad y autor), para decidir si quiero cocinarla.

Criterios de aceptación: la vista muestra la imagen principal, metadatos completos, lista de ingredientes con cantidades, pasos numerados y datos del autor; el botón de like refleja el estado actual del usuario; el botón Start Cooking lleva al modo cocina guiado.

HU-06 — BUSCAR RECETAS Y USUARIOS

Como usuario, quiero buscar recetas por título y usuarios por nombre en tiempo real, para descubrir contenido y personas que me interesen.

Criterios de aceptación: los resultados se actualizan en tiempo real mientras el usuario escribe; se muestran tanto recetas como usuarios que coincidan con el término de búsqueda; si no hay resultados se muestra un mensaje informativo.

Interacción social

HU-07 — DAR LIKE A UNA RECETA

Como usuario autenticado, quiero dar like a una receta que me guste, para mostrar mi apreciación al autor.

Criterios de aceptación: el icono de corazón cambia de estado inmediatamente al pulsar; el contador de likes se recalcula localmente de forma inmediata sin esperar respuesta del servidor; cada usuario solo puede dar un like por receta; al volver a pulsar se quita el like.

HU-08 — SEGUIR A UN USUARIO

Como usuario autenticado, quiero seguir a otros usuarios, para ver sus recetas en mi feed y estar al día de su actividad.

Criterios de aceptación: el botón Follow/Following cambia de estado visualmente de forma inmediata; el contador de seguidores se actualiza localmente al instante; el usuario puede dejar de seguir pulsando de nuevo el botón; el botón de seguir no aparece en el perfil propio del usuario.

Modo cocina y gamificación

HU-09 — USAR LA COCINA EN MODO GUIADO

Como usuario autenticado, quiero seguir los pasos de una receta con un temporizador integrado, para cocinar de forma guiada sin perder el hilo.

Criterios de aceptación: los pasos se muestran de uno en uno con su número y texto; hay un temporizador global en segundos que puede pausarse y reanudarse; la barra de progreso refleja el avance en todo momento; al pulsar Finish Cooking en el último paso se registra la receta como cocinada junto con el tiempo transcurrido.

HU-10 — VER MIS LOGROS OBTENIDOS

Como usuario autenticado, quiero ver los logros que he desbloqueado en mi perfil, para conocer mi progreso y sentirme motivado a seguir participando.

Criterios de aceptación: la pestaña Achievements del perfil muestra los logros desbloqueados con icono, nombre, descripción y puntos; los logros se desbloquean automáticamente en el backend cuando el usuario alcanza los umbrales definidos (recetas publicadas, likes recibidos, recetas cocinadas, seguidores obtenidos).

Administración

HU-11 — CONSULTAR LAS ESTADÍSTICAS DE LA PLATAFORMA

Como administrador, quiero consultar las estadísticas globales de la plataforma (usuarios, recetas, likes, comentarios y gráficos de evolución), para tomar decisiones informadas sobre el crecimiento del servicio.

Criterios de aceptación: el panel carga los KPIs al acceder a la vista; los datos incluyen gráficos de distribución por categoría, evolución mensual de usuarios y ranking de usuarios más activos; el panel solo es accesible para usuarios con rol ADMIN; los demás usuarios son redirigidos si intentan acceder.

6. Desarrollo técnico detallado

6.1 Metodología de trabajo

La metodología seguida en el desarrollo de PlateUp fue iterativa e incremental. El proyecto no se abordó de forma monolítica, sino que se construyó de manera progresiva, añadiendo funcionalidades completas en cada iteración y garantizando siempre una versión funcional del sistema sobre la que continuar trabajando. Este enfoque es similar a la metodología ágil Scrum, aunque adaptado a un equipo de una sola persona y sin sprints formales definidos.

Cada iteración seguía el mismo ciclo: análisis de la funcionalidad a implementar, diseño de la solución técnica, implementación en el backend, prueba de los endpoints con Swagger UI, implementación en el frontend, integración y prueba end-to-end, y commit al repositorio con un mensaje descriptivo del trabajo realizado.

El control de versiones con Git fue fundamental para la gestión del desarrollo. Los commits frecuentes con mensajes claros permitieron tener un historial detallado de la evolución del proyecto, facilitar la reversión de cambios problemáticos y mantener una copia de seguridad remota siempre actualizada en GitHub.

6.2 Desarrollo del backend (Spring Boot + API REST)

Estructura del proyecto

El proyecto backend de PlateUp sigue la estructura de paquetes estándar de Spring Boot, con cada capa claramente separada:

- `es.ieslavereda.plateup.model`: Entidades JPA que representan las tablas de la base de datos. Cada clase está anotada con `@Entity` y `@Table(name=...)` para definir la correspondencia con la tabla SQL. Las relaciones entre entidades se declaran con `@OneToMany`, `@ManyToOne`, `@ManyToMany` y `@EmbeddedId` para las claves compuestas.
- `es.ieslavereda.plateup.repository`: Interfaces que extienden `JpaRepository<Entidad, TipoClave>` para cada entidad. Spring Data JPA genera automáticamente la implementación en tiempo de ejecución. Incluyen métodos de consulta derivados del nombre (`findByUsername`, `findByRecipeId`, `existsByFollowerIdAndFollowedId`, etc.).
- `es.ieslavereda.plateup.service`: Clases `@Service` con la lógica de negocio de la aplicación. `AchievementUnlockService` gestiona el desbloqueo automático de logros. `FileStorageService` gestiona la subida y almacenamiento de archivos de imagen.
- `es.ieslavereda.plateup.controller`: Clases `@RestController` con `@RequestMapping` que gestionan las peticiones HTTP y delegan en los repositorios y servicios. Cada controlador responde con `ResponseEntity` y los códigos HTTP apropiados.
- `es.ieslavereda.plateup.security`: `JwtService` genera y valida tokens JWT. `JwtAuthenticationFilter` intercepta las peticiones y establece el contexto de seguridad.
- `es.ieslavereda.plateup.config`: `SecurityConfig` configura Spring Security. `WebConfig` configura los recursos estáticos. `AchievementSyncRunner` se ejecuta al arrancar el servidor para sincronizar los logros definidos.

- `es.ieslavereda.plateup.dto`: `LoginRequest` y `RegisterRequest` son los DTOs para autenticación. `AuthResponse` devuelve el token JWT y los datos del usuario tras el login.
- `es.ieslavereda.plateup.exception`: `GlobalExceptionHandler` maneja las excepciones de forma centralizada con `@ControllerAdvice`. `ResourceNotFoundException` y `AuthenticationRequiredException` son las excepciones personalizadas.

Controladores REST implementados

PlateUp cuenta con 18 controladores REST que cubren todas las entidades del sistema:

Controlador	Funcionalidad
UserController	Registro, login, obtención de perfil, actualización de datos, búsqueda de usuarios, listado de seguidores y seguidos.
RecipeController	CRUD de recetas, listado paginado, filtrado por categoría, búsqueda por título, recetas de un usuario.
RecipeStepController	CRUD de pasos de receta asociados a una receta concreta.
RecipeIngredientController	Gestión de ingredientes con cantidad y unidad asociados a cada receta.
IngredientController	CRUD del catálogo de ingredientes del sistema.
LikeController	Dar like, quitar like, contar likes de una receta, verificar si el usuario actual ha dado like.
FollowController	Seguir usuario, dejar de seguir, listar seguidores, listar seguidos, verificar estado de seguimiento.
AchievementController	Listado de todos los logros disponibles en el sistema.
UserAchievementController	Logros obtenidos por un usuario, asignación de nuevos logros.
ChallengeController	CRUD de retos del sistema.
UserChallengeController	Progreso de cada usuario en los retos activos.
CollectionController	CRUD de colecciones de recetas de cada usuario.
CollectionRecipeController	Gestión de recetas dentro de cada colección.
CommentController	CRUD de comentarios asociados a recetas.
CookedRecipeController	Registro de recetas cocinadas por cada usuario con tiempo transcurrido.
UtensilController	CRUD del catálogo de utensilios de cocina.
UploadController	Subida de imágenes de recetas y avatares de usuario.
AdminController	Estadísticas globales de la plataforma para el panel de administración.

Documentación automática con Springdoc OpenAPI

El proyecto incluye la dependencia `springdoc-openapi-starter-webmvc-ui` en el `pom.xml`, que genera automáticamente la documentación interactiva de la API REST en formato Swagger UI. Durante el desarrollo, la documentación está disponible en `http://localhost:8080/swagger-ui/index.html` y permite explorar y probar todos los endpoints directamente desde el navegador. Este endpoint está configurado como público en Spring Security para facilitar el acceso durante el desarrollo.

6.3 Desarrollo del frontend (Vue.js 3)

Estructura del proyecto frontend

El proyecto frontend se organiza en las siguientes carpetas dentro de `src/`:

- `views/`: Las 11 vistas principales de la aplicación (`HomeView`, `ExploreView`, `CreateView`, `ProfileView`, `EditProfileView`, `RecipeDetailView`, `UserProfileView`, `ConnectionsView`, `CookingView`, `AuthView`, `AdminDashboardView`). Cada vista es un componente Vue SFC con su lógica, template y estilos.
- `components/`: Componentes reutilizables. `BottomNav.vue` implementa la barra de navegación inferior persistente. `RecipeCard.vue` es el componente de tarjeta de receta reutilizado en el feed y el perfil.
- `router/index.js`: Configuración completa del enrutador Vue Router con todas las rutas, guards de navegación (autenticación obligatoria, guard `adminOnly`) y redireccionamientos.
- `stores/`: `authStore.js` gestiona el estado de autenticación con Pinia (token JWT, datos del usuario, métodos `initialize`, `login`, `logout`). `index.js` crea la instancia de Pinia.
- `services/`: Un archivo de servicio por cada entidad del sistema (`userService.js`, `recipeService.js`, `likeService.js`, `followService.js`, etc.). Cada servicio encapsula las llamadas a la API REST mediante Axios.
- `utils/`: Funciones de utilidad reutilizables. `recipeCategories.js` define las 14 categorías y su función de normalización. `media.js` gestiona las URLs de imágenes. `userAvatar.js` gestiona los avatares de usuario. `recipeImage.js` gestiona las imágenes de receta con fallback.
- `assets/`: Archivos estáticos. `main.css` define los estilos globales con las directivas de Tailwind CSS v4.

Interceptor de Axios

El archivo `src/services/api.js` configura la instancia global de Axios con la URL base de la API (configurada mediante la variable de entorno `VITE_API_URL`). Además, implementa un interceptor de peticiones que añade automáticamente la cabecera `Authorization: Bearer {token}` a todas las peticiones, leyendo el token del `localStorage`. Esto significa que ningún servicio necesita gestionar manualmente la autenticación; simplemente llama a la función correspondiente de su servicio y el interceptor se encarga del resto.

Guard de navegación en Vue Router

El archivo `router/index.js` implementa un `beforeEach` global que se ejecuta antes de cada navegación. Este guard verifica si la ruta requiere autenticación (todas excepto `/login`), y si el usuario no está autenticado, lo redirige a la pantalla de login. También verifica si la ruta tiene la meta `adminOnly` y, si el usuario no tiene permisos de administrador, lo redirige a la pantalla principal. La raíz `/` redirige al perfil si el usuario está autenticado o al login si no lo está.

6.4 Aplicación móvil Android con Capacitor

La versión Android de PlateUp se desarrolló utilizando Capacitor 8, que transforma la aplicación Vue.js en una aplicación Android instalable. El proceso de desarrollo siguió estos pasos:

- Configuración de Capacitor en el proyecto Vue.js mediante el archivo `capacitor.config.json`, que define el identificador del paquete (`com.plateup.app`), el nombre de la aplicación y la URL del servidor web local para el modo de desarrollo.
- Construcción del proyecto Vue.js con el comando `npm run build`, que genera los archivos estáticos optimizados en el directorio `dist/`.
- Sincronización del directorio `dist/` con el proyecto Android mediante el comando `npx cap sync android`, que copia los archivos web al directorio `android/app/src/main/assets/public/`.
- Apertura del proyecto en Android Studio con `npx cap open android` para compilar el APK y ejecutarlo en un dispositivo o emulador Android.
- Configuración del `AndroidManifest.xml` con los permisos necesarios: acceso a internet (`INTERNET`), acceso al almacenamiento de archivos (`READ_EXTERNAL_STORAGE`, `WRITE_EXTERNAL_STORAGE`).

El archivo `capacitor.config.json` configura el comportamiento de la aplicación en Android, incluyendo la política de uso de cámara para la subida de imágenes de recetas y avatares desde la galería.

6.5 Sistema de subida de imágenes

PlateUp implementa un sistema de subida de imágenes gestionado por `FileStorageService` en el backend y `UploadController` que expone el endpoint `POST /api/upload`. El frontend utiliza el componente de input tipo `file` para seleccionar la imagen, la previsualiza localmente con `URL.createObjectURL()`, y la envía al servidor en formato `multipart/form-data` mediante `Axios`.

El servidor recibe el archivo, valida el tipo MIME y el tamaño (máximo 5 MB configurado en `application.properties`), genera un nombre de archivo único basado en UUID para evitar colisiones, y lo almacena en el directorio `uploads/` configurado mediante la variable de entorno `APP_UPLOAD_DIR`. La URL pública del archivo (`/uploads/{nombre_archivo}`) se devuelve al frontend, que la almacena en el campo `image_url` de la receta o en `avatar_url` del usuario.

6.6 Sistema de gamificación

El sistema de gamificación de PlateUp se basa en dos elementos complementarios: logros (`Achievements`) y retos (`Challenges`). Los logros son hitos permanentes que el usuario desbloquea al cumplir determinadas condiciones (publicar su primera receta, alcanzar 10 seguidores, cocinar 5 recetas, etc.). El sistema de logros está completamente funcional: `AchievementSyncRunner` se ejecuta automáticamente al arrancar el servidor sincronizando los logros definidos en el código, y `AchievementUnlockService` evalúa automáticamente las condiciones de desbloqueo cada vez que el usuario realiza una acción relevante en los controladores de `RecipeController`, `FollowController` y `CookedRecipeController`.

Los retos son desafíos temporales con fecha de inicio y fin, objetivo cuantificable y puntos de recompensa que permiten al usuario participar en competiciones de corta duración dentro de la plataforma. La estructura de datos para los retos está completamente implementada: la tabla `Challenges` almacena los retos del sistema con nombre, descripción, fechas y objetivo,

mientras que UserChallenges registra el progreso individual de cada usuario en cada reto (progreso actual, estado: in_progress/completed/abandoned, fechas de participación). Los endpoints REST para gestión de retos están desarrollados y funcionales para consultar retos disponibles, asignar retos a usuarios y actualizar el progreso manualmente. La automatización del seguimiento de progreso de retos está diseñada para implementación futura: requeriría crear un servicio ChallengeProgressService que se integre en los puntos clave donde el usuario realiza acciones cuantificables (publicar receta, cocinar receta, seguir usuario), actualizando automáticamente el progreso y marcando retos como completados cuando se alcanza el objetivo. Esta estructura modular permite que la funcionalidad de retos pueda activarse fácilmente en versiones posteriores de PlateUp sin cambios en la arquitectura actual.

AchievementSyncRunner es un CommandLineRunner de Spring Boot que se ejecuta automáticamente cada vez que arranca el servidor, sincronizando los logros definidos en el código con los registros de la tabla Achievements en la base de datos. Esto garantiza que los logros del sistema siempre estén actualizados sin necesidad de ejecutar scripts SQL manualmente.

AchievementUnlockService es el servicio que evalúa las condiciones de desbloqueo de cada logro para un usuario dado. Se invoca desde los controladores relevantes (RecipeController, FollowController, CookedRecipeController) cada vez que un usuario realiza una acción que podría desbloquear un logro.

6.7 Pruebas realizadas

Las pruebas del sistema se realizaron de forma manual y sistemática en dos entornos: local (con el servidor Spring Boot corriendo en localhost:8080 y la base de datos MySQL local) y producción (con el backend en Railway y la base de datos en Railway).

Pruebas de la API REST

Cada endpoint de la API se probó con Swagger UI, verificando los códigos de respuesta HTTP correctos (200 OK, 201 Created, 400 Bad Request, 401 Unauthorized, 404 Not Found), los cuerpos de respuesta JSON esperados, y el comportamiento correcto de la autenticación JWT (acceso correcto con token válido, rechazo con token inválido o expirado, rechazo sin token en endpoints protegidos).

Pruebas de la aplicación web

Se probaron manualmente todos los flujos de usuario: registro de cuenta nueva, inicio de sesión, creación de receta completa con ingredientes y pasos, subida de imagen, dar like y quitarlo, seguir a un usuario y dejar de seguirlo, navegar entre vistas, editar el perfil, cambiar el avatar, acceder al modo cocina y completar una receta. Se verificó el comportamiento en Chrome, Firefox y Edge.

Pruebas de la aplicación Android

La versión Android se probó en un dispositivo físico Android real, verificando que la interfaz se visualiza correctamente en pantalla móvil, que la navegación funciona con gestos táctiles, que la subida de imágenes desde la galería funciona correctamente, y que la comunicación con la API de producción en Railway funciona sin problemas.

Pruebas con usuarios reales

Además de las pruebas técnicas, la aplicación se presentó a usuarios reales (familiares y conocidos) para que actuaran como testers independientes. Este feedback fue invaluable para identificar errores de usabilidad, flujos de navegación no intuitivos y casos límite que no habían sido considerados durante el desarrollo. Las observaciones de usuarios sin contexto técnico previo revelaron asunciones implícitas en el diseño: botones confusos, mensajes de error poco claros, información faltante en pantallas. Cada sesión de testing resultó en correcciones que mejoraron significativamente la experiencia final de la aplicación.

6.8 Despliegue en producción (Railway)

El despliegue en producción de PlateUp se realizó utilizando el servicio de cloud de Railway.

Base de datos MySQL gestionada

Railway es una plataforma cloud de despliegue que permite gestionar tanto servicios de aplicación como bases de datos en un único entorno. Se eligió Railway para la base de datos de PlateUp por varias razones: centraliza el despliegue del backend y la base de datos en una misma plataforma, ofrece una interfaz sencilla para gestionar servicios MySQL, y proporciona conexión SSL cifrada y variables de entorno gestionadas de forma segura.

La configuración consiste en crear un servicio MySQL en Railway, obtener la cadena de conexión JDBC con host, puerto, nombre de base de datos, usuario y contraseña, y configurarla como variable de entorno en Railway. Railway proporciona también una interfaz web para explorar los datos de la base de datos directamente desde el navegador, lo cual facilita la depuración durante el desarrollo.

Backend (API REST)

Railway también permite desplegar aplicaciones web, APIs, workers y sitios estáticos directamente desde un repositorio GitHub. El backend de PlateUp se despliega en Railway como un Web Service Docker, usando el Dockerfile incluido en el directorio plateup/. El archivo railway.toml en la raíz del repositorio define la configuración completa del servicio: nombre, runtime Docker, ruta del Dockerfile, contexto de construcción y variables de entorno necesarias.

Frontend (Sitio estático)

El frontend de PlateUp se despliega como sitio web estático en Railway, accesible desde cualquier dispositivo con conexión a internet, lo que facilita la demostración visual y la revisión de cambios sin necesidad de reinstalar la APK. Adicionalmente, los archivos generados por npm run build se sincronizan con el proyecto Android mediante npx cap sync android y se empaquetan desde Android Studio para su distribución como aplicación nativa. En ambos casos, la variable de entorno VITE_API_URL apunta a la URL de producción del backend, garantizando la comunicación correcta con la API.

Docker Compose para desarrollo local

Para facilitar el desarrollo local sin necesidad de tener Railway configurado, el repositorio incluye un archivo docker-compose.yml que levanta tres contenedores: una imagen oficial de MySQL 8.0 con el charset utf8mb4, el backend Spring Boot compilado desde el Dockerfile local, y el frontend Vue.js servido con Nginx. Los contenedores se comunican en una red Docker interna, y el backend espera a que la base de datos esté disponible gracias a la

configuración de healthcheck y depends_on. El script db/Pruebas.sql se monta como volumen de inicialización de MySQL para crear el esquema y los datos de prueba automáticamente.

7. Base de datos

7.1 Diseño conceptual

El diseño de la base de datos de PlateUp parte de un análisis exhaustivo de las funcionalidades de la aplicación. El modelo de datos debe dar soporte a: la gestión de usuarios con perfil completo, la creación y consulta de recetas con todos sus componentes (pasos, ingredientes, utensilios), las interacciones sociales (likes y follows), las colecciones de recetas, el sistema de gamificación (logros y retos), el modo cocina (registro de recetas cocinadas) y las funciones de administración (reportes, auditoría).

Se adoptó un modelo relacional normalizado para eliminar redundancias y garantizar la integridad de los datos. Las relaciones muchos a muchos (como Usuarios-Recetas en likes, o Recetas-Ingredientes) se implementan mediante tablas intermedias con claves compuestas.

El charset elegido para toda la base de datos es utf8mb4 con collation utf8mb4_unicode_ci, que soporta el rango completo de caracteres Unicode incluyendo emojis, necesarios para los textos de recetas y los iconos de logros.

7.2 Descripción de las entidades

Users — Usuarios

Tabla central del sistema. Almacena la información de cada usuario registrado: identificador único auto-incremental, username y email con restricción UNIQUE, hash de contraseña BCrypt, nombre para mostrar (display_name), biografía (bio), URL del avatar, visibilidad por defecto de las recetas (privada, solo seguidores o pública) y fechas de creación y modificación con actualización automática.

Recipes — Recetas

Almacena cada receta publicada en la plataforma. Incluye referencia al usuario autor (user_id con FK a Users y ON DELETE CASCADE), título, descripción, URL de imagen, número de raciones, tiempo total en minutos, nivel de dificultad (enum: easy, medium, hard) y timestamps de creación y modificación. El ON DELETE CASCADE garantiza que si se elimina un usuario, todas sus recetas se eliminan en cascada.

RecipeSteps — Pasos de receta

Cada fila representa un paso de preparación de una receta. Incluye el índice de orden (order_index) para mantener la secuencia correcta, el texto descriptivo del paso, el tiempo de temporizador en segundos (timer_seconds, opcional) y una URL de media asociada al paso (opcional). Se relaciona con Recipes mediante FK con ON DELETE CASCADE.

Ingredients — Ingredientes

Catálogo global de ingredientes del sistema que crece de forma orgánica con el uso. Los ingredientes se gestionan mediante esta tabla centralizada: cuando el usuario añade ingredientes a una receta, puede seleccionar del catálogo existente o crear nuevos si no encuentra lo que busca, permitiendo que el catálogo se enriquezca constantemente con nuevas opciones. Cada ingrediente tiene un nombre único (evitando duplicidades) y una

unidad por defecto que facilita la consistencia en las medidas. Este catálogo compartido por todas las recetas evita la fragmentación de datos y mejora la experiencia del usuario al proporcionar un conjunto de ingredientes validados y consistentes.

RecipeIngredients — Ingredientes de receta

Tabla intermedia entre Recipes e Ingredients. Almacena la cantidad decimal del ingrediente en la receta, la unidad de medida utilizada (que puede diferir de la unidad por defecto del ingrediente) y notas adicionales (por ejemplo, 'tamizado', 'en rodajas', etc.).

Utensils y RecipeUtensils — Utensilios

Catálogo global de utensilios de cocina (Utensils) y tabla de relación muchos a muchos con recetas (RecipeUtensils) con clave primaria compuesta (recipe_id, utensil_id).

Follows — Seguimientos

Tabla de relaciones de seguimiento entre usuarios. Contiene follower_id (quien sigue) y followed_id (a quien se sigue), ambos con FK a Users. Incluye un campo status (enum: pending, active) para un posible sistema de seguimiento privado, y la fecha de inicio del seguimiento.

Likes — Me gusta

Tabla de likes entre usuarios y recetas. La clave primaria compuesta (user_id, recipe_id) garantiza que cada usuario solo puede dar un like por receta. Registra la fecha del like.

Collections y CollectionRecipes — Colecciones

Los usuarios pueden organizar recetas en colecciones personales (Collections), que pueden ser públicas o privadas. CollectionRecipes es la tabla intermedia entre Collections y Recipes, con un campo order_index para mantener el orden de las recetas dentro de la colección.

Achievements y UserAchievements — Logros

Achievements define el catálogo de logros disponibles: nombre, descripción, URL de icono, categoría, puntos y timestamp de creación. UserAchievements registra los logros obtenidos por cada usuario con la fecha de obtención y el nivel alcanzado.

Challenges y UserChallenges — Retos

Challenges define los retos del sistema con nombre, descripción, fechas de inicio y fin, objetivo numérico y puntos de recompensa. UserChallenges registra el progreso de cada usuario (campo progress), el estado (enum: in_progress, completed, abandoned) y las fechas de participación. La estructura está completamente implementada con endpoints REST funcionales. La automatización del progreso está diseñada para implementación futura mediante un servicio que actualice automáticamente el progreso cuando el usuario realiza acciones cuantificables.

CookedRecipes — Recetas cocinadas

Registro de las veces que un usuario ha cocinado una receta, con la fecha de cocción y el tiempo transcurrido en segundos (medido por el temporizador global del modo cocina).

Tablas adicionales del sistema

El esquema incluye también: Comments (comentarios de usuarios sobre recetas), Media (archivos multimedia asociados a recetas o pasos), Notifications (notificaciones del sistema para los usuarios), Reports (reportes de contenido inapropiado), y AuditLogs (registro de acciones de administración del sistema). Estas tablas están definidas en el esquema para uso futuro y escalabilidad del sistema.

7.3 Paso a tablas (modelo relacional completo)

Tabla	Definición de campos
USERS	id BIGINT PK AI, username VARCHAR(50) UNIQUE NOT NULL, email VARCHAR(100) UNIQUE NOT NULL, password_hash VARCHAR(255) NOT NULL, display_name VARCHAR(100), bio TEXT, avatar_url VARCHAR(255), visibility_default ENUM('private','followers','public') DEFAULT 'public', created_at DATETIME, updated_at DATETIME
RECIPES	id BIGINT PK AI, user_id BIGINT FK→Users ON DELETE CASCADE, title VARCHAR(150) NOT NULL, description TEXT, image_url VARCHAR(500), servings INT, total_minutes INT, difficulty ENUM('easy','medium','hard'), created_at DATETIME, updated_at DATETIME
RECIPESTEPS	id BIGINT PK AI, recipe_id BIGINT FK→Recipes ON DELETE CASCADE, order_index INT NOT NULL, text TEXT NOT NULL, timer_seconds INT, media_url VARCHAR(255)
INGREDIENTS	id BIGINT PK AI, name VARCHAR(100) UNIQUE NOT NULL, unit_default VARCHAR(20)
RECIPEINGREDIENTS	id BIGINT PK AI, recipe_id BIGINT FK→Recipes ON DELETE CASCADE, ingredient_id BIGINT FK→Ingredients ON DELETE CASCADE, quantity_decimal DECIMAL(10,2), unit VARCHAR(20), notes TEXT
UTENSILS	id BIGINT PK AI, name VARCHAR(100) UNIQUE NOT NULL
RECIPEUTENSILS	recipe_id BIGINT FK→Recipes ON DELETE CASCADE, utensil_id BIGINT FK→Utensils ON DELETE CASCADE, PK(recipe_id, utensil_id)
FOLLOWS	follower_id BIGINT FK→Users ON DELETE CASCADE, followed_id BIGINT FK→Users ON DELETE CASCADE, status ENUM('pending','active') DEFAULT 'active', created_at DATETIME, PK(follower_id, followed_id)
LIKES	user_id BIGINT FK→Users ON DELETE CASCADE, recipe_id BIGINT FK→Recipes ON DELETE CASCADE, created_at DATETIME, PK(user_id, recipe_id)
COLLECTIONS	id BIGINT PK AI, user_id BIGINT FK→Users ON DELETE CASCADE, name VARCHAR(100) NOT NULL, is_public TINYINT(1) DEFAULT 1, created_at DATETIME

COLLECTIONRECIPES	collection_id BIGINT FK→Collections ON DELETE CASCADE, recipe_id BIGINT FK→Recipes ON DELETE CASCADE, order_index INT, PK(collection_id, recipe_id)
ACHIEVEMENTS	id BIGINT PK AI, name VARCHAR(100) NOT NULL, description TEXT, icon_url VARCHAR(255), category VARCHAR(50), points INT DEFAULT 0, created_at DATETIME
USERACHIEVEMENTS	user_id BIGINT FK→Users ON DELETE CASCADE, achievement_id BIGINT FK→Achievements ON DELETE CASCADE, earned_at DATETIME, level INT DEFAULT 1, PK(user_id, achievement_id)
CHALLENGES	id BIGINT PK AI, name VARCHAR(150) NOT NULL, description TEXT, start_date DATETIME, end_date DATETIME, goal INT, reward_points INT DEFAULT 0
USERCHALLENGES	user_id BIGINT FK→Users ON DELETE CASCADE, challenge_id BIGINT FK→Challenges ON DELETE CASCADE, progress INT DEFAULT 0, status ENUM('in_progress','completed','abandoned') DEFAULT 'in_progress', started_at DATETIME, finished_at DATETIME, PK(user_id, challenge_id)
COMMENTS	id BIGINT PK AI, user_id BIGINT FK→Users ON DELETE CASCADE, recipe_id BIGINT FK→Recipes ON DELETE CASCADE, text TEXT NOT NULL, created_at DATETIME
COOKEDRECIPES	id BIGINT PK AI, user_id BIGINT FK→Users ON DELETE CASCADE, recipe_id BIGINT FK→Recipes ON DELETE CASCADE, cooked_at DATETIME, elapsed_seconds INT

8. Diseño visual del proyecto (UX/UI)

8.1 Filosofía de diseño y decisiones generales

El diseño de PlateUp sigue una filosofía mobile-first, partiendo de la interfaz más pequeña (dispositivo Android de 360-414px) y escalando hacia el entorno de escritorio. Todas las decisiones de diseño están orientadas a maximizar la usabilidad en pantallas táctiles pequeñas: elementos interactivos con área de toque generosa (mínimo 44x44px), navegación accesible con el pulgar, cards con bordes redondeados y sin esquinas afiladas, y uso generoso del espacio en blanco para evitar la sensación de aglomeración.

La paleta de colores se basa en un color principal rojizo-naranja (#f45b3f) para las acciones primarias y el contenido destacado, fondos muy claros (#f6f3ee) para reducir la fatiga visual, tarjetas blancas con sombra muy suave para separar el contenido, y acentos naranjas suaves (orange-50) para los estados de hover y las secciones de información secundaria. Esta paleta transmite calidez y energía, coherente con el ámbito de la cocina.

La tipografía utiliza la fuente del sistema operativo (system-ui) con pesos semibold y bold para títulos y elementos de navegación, y regular para el texto de contenido. Los tamaños varían entre 12px (etiquetas secundarias), 14-16px (cuerpo de texto) y 24-32px (títulos de sección).

La navegación principal se organiza en una barra inferior fija (BottomNav) con cuatro iconos: Home (feed principal), Explore (descubrimiento), Create (nueva receta) y Profile (perfil propio). Esta barra es persistente en todas las pantallas principales y ofrece acceso inmediato a cualquier sección desde cualquier punto de la aplicación.

8.2 AuthView — Pantalla de autenticación

La pantalla de autenticación es el punto de entrada a la aplicación para usuarios no registrados. Presenta un diseño limpio y centrado con el logotipo de PlateUp en la parte superior, un formulario compacto en el centro y la opción de alternar entre el modo de inicio de sesión y el de registro.

En modo inicio de sesión, el formulario solicita username y contraseña. En modo registro, los campos son username, email y contraseña. El cambio entre modos se realiza con una transición suave sin recargar la página. Ambos formularios incluyen validación del lado del cliente antes de enviar los datos al servidor, y muestran mensajes de error descriptivos en caso de fallo (credenciales incorrectas, usuario ya existente, email ya registrado).

Tras un inicio de sesión exitoso, el token JWT recibido se almacena en el store de Pinia y el router redirige automáticamente al usuario a la pantalla de perfil. Tras un registro exitoso, el usuario queda autenticado directamente y accede a la aplicación sin necesidad de un segundo paso de verificación.

8.3 HomeView — Feed principal

La pantalla Home es el núcleo de la experiencia social de PlateUp. Se divide en tres secciones principales que se organizan verticalmente en una vista con scroll:

Encabezado

El encabezado muestra el saludo 'Welcome back' y el título 'PlateUp' a la izquierda, y el avatar del usuario actual a la derecha como botón que navega directamente al perfil del usuario. Este acceso rápido al perfil propio desde la pantalla principal es un patrón de diseño habitual en las redes sociales más populares.

Filtros de categoría

Una fila horizontal con scroll horizontal presenta los 14 filtros de categoría disponibles más la opción 'All' para ver todas las recetas. El filtro activo se destaca con el color principal (#f45b3f) y texto blanco. Al tocar un filtro, el feed se actualiza automáticamente para mostrar únicamente las recetas de esa categoría. El componente gestiona la reactividad mediante las referencias de Vue 3 y las propiedades computadas.

Panel de progreso culinario (Cooking Journey)

Una tarjeta destacada con gradiente del color principal (de #f45b3f a #ff8a66) muestra el progreso personal del usuario. En la cabecera de la tarjeta se muestra el mensaje motivacional 'Keep your streak alive' y la racha de días consecutivos de actividad. Debajo, tres métricas en tarjetas translúcidas: número de recetas publicadas, número de medallas (logros obtenidos) y puntos totales acumulados. Este panel gamifica la experiencia del usuario y le proporciona un incentivo constante para seguir participando.

Feed de recetas

El feed muestra las recetas ordenadas por fecha de publicación (más recientes primero), filtradas por la categoría seleccionada si corresponde. Cada receta se muestra en un RecipeCard que incluye: imagen de la receta con bordes redondeados, dificultad y tiempo de preparación en etiquetas de color, categoría de la receta, nombre y avatar del autor (con acceso al perfil del autor al tocarlo), título de la receta, descripción truncada a dos líneas, y el contador de likes con el icono de corazón. El usuario puede dar like directamente desde la tarjeta sin necesidad de entrar al detalle de la receta. Al tocar cualquier otra parte de la tarjeta, navega al detalle completo de la receta.

8.4 ExploreView — Exploración y búsqueda

La pantalla Explore está diseñada para el descubrimiento de contenido y personas. Se organiza en tres secciones verticales:

Barra de búsqueda

Una barra de búsqueda prominente con icono de lupa permite buscar tanto recetas como usuarios. La búsqueda es reactiva: a medida que el usuario escribe, el sistema filtra los resultados en tiempo real consultando la API. Los resultados se muestran diferenciados: si el término de búsqueda coincide con nombres de usuario, aparece la lista de usuarios encontrados; si coincide con títulos de recetas, aparecen las recetas.

Personas para descubrir (People to discover)

Una lista horizontal con scroll de tarjetas de usuario sugiere a las personas más activas o relevantes de la plataforma. Cada tarjeta muestra el avatar del usuario en circular con un ring naranja, el nombre para mostrar (display_name), el username con el símbolo @, y un botón de Follow/Following que permite seguir o dejar de seguir al usuario directamente desde esta pantalla. El botón cambia de color y texto según el estado actual del seguimiento (naranja

'Follow' si no se sigue, gris 'Following' si ya se sigue). Esta sección facilita el crecimiento de la red social de cada usuario.

Grid de recetas

Un grid de dos columnas muestra las recetas de la plataforma en formato cuadrado con imagen de fondo, similar al grid de Instagram. Al tocar una receta del grid, navega al detalle de esa receta. Este formato permite explorar mucho contenido visual en poco espacio, priorizando la imagen de la comida como elemento principal de atracción.

8.5 RecipeDetailView — Detalle de receta

La pantalla de detalle de receta es una de las más ricas en contenido de toda la aplicación. Se organiza en las siguientes secciones:

Hero con imagen y acciones

La parte superior ocupa un 40% de la pantalla con la imagen principal de la receta en formato panorámico, con un gradiente oscuro en la parte inferior sobre el que se superponen el título y la descripción de la receta en texto blanco. Un botón 'Back' en la esquina superior izquierda permite retroceder. En la esquina inferior derecha, el botón de like muestra el contador de likes y cambia a color naranja relleno cuando el usuario ha dado like. La categoría de la receta aparece como una etiqueta redondeada blanca superpuesta sobre la imagen.

Metadatos de la receta

Debajo de la imagen, una fila de etiquetas muestra: el nivel de dificultad (Easy, Medium o Hard en naranja), el tiempo total de preparación en minutos, el número de raciones y la categoría. A continuación, la información del autor: avatar circular y nombre del usuario que publicó la receta, con acceso a su perfil al tocarlo.

Navegación por secciones

La receta se organiza en tres secciones de contenido que se muestran consecutivamente de forma vertical: ingredientes, pasos de preparación y un botón destacado "Start Cooking" para acceder al modo cocina guiado. Cada sección está contenida en una tarjeta blanca con bordes redondeados.

Sección de ingredientes

Muestra la lista completa de ingredientes con cantidad, unidad y nombre. Cada ingrediente incluye un checkbox que el usuario puede marcar para ir registrando los ingredientes que ya tiene preparados antes de cocinar. Los ingredientes marcados se muestran marcados para facilitar el seguimiento visual.

Sección de pasos

Muestra los pasos de preparación numerados. Cada paso muestra su número de orden en un círculo naranja, el texto descriptivo. El botón 'Start Cooking' naranja al pie de esta sección inicia el modo cocina guiado.

8.6 CookingView — Modo cocina guiado

El modo cocina es una de las funcionalidades más diferenciadas de PlateUp. Es una vista optimizada para su uso mientras se cocina, con una interfaz simplificada y elementos de gran tamaño para facilitar la interacción con las manos mojadas o enharinadas.

Temporizador global

En la parte superior, un temporizador global muestra el tiempo transcurrido desde el inicio de la sesión de cocina en formato MM:SS. Dos botones permiten pausar y reanudar el temporizador. El temporizador se muestra en naranja cuando está activo y en gris cuando está pausado. Al finalizar la receta, el tiempo total se registra en la tabla CookedRecipes.

Navegación entre pasos

Los pasos de la receta se muestran de uno en uno, a pantalla completa, con el número del paso actual. Una barra de progreso horizontal en la parte superior muestra visualmente el avance.

Finalización de la receta

Al llegar al último paso, aparece un botón 'Finish Cooking' que marca la receta como cocinada, registra el tiempo total en la base de datos y navega de vuelta al detalle de la receta con un mensaje de felicitación.

8.7 CreateView — Creación de receta

La pantalla de creación de recetas es el formulario más complejo de la aplicación. Se organiza en varias secciones dentro de un formulario con validación en tiempo real:

Información básica

Campos de texto para el título y la descripción de la receta. El título tiene una longitud máxima de 150 caracteres, coherente con la restricción de la base de datos. La descripción es un textarea de varias líneas con placeholder descriptivo.

Imagen de la receta

Un área de previsualización de imagen que inicialmente muestra un placeholder gráfico. Los botones 'Choose image' y 'Remove' permiten seleccionar una imagen desde la galería del dispositivo o eliminar la imagen seleccionada. La imagen se previsualiza inmediatamente en el área correspondiente antes de publicar. El archivo se envía al backend mediante FormData en el momento de publicar la receta.

Metadatos de la receta

Un selector de categoría (dropdown) con las 14 categorías disponibles. Un selector de dificultad (Easy, Medium, Hard) en formato de botones de radio visuales. Campos numéricos para el tiempo de preparación en minutos y el número de raciones.

Ingredientes

Una sección dinámica donde el usuario añade ingredientes uno a uno. Cada ingrediente se añade con nombre, cantidad y unidad de medida. Un botón 'Add ingredient' añade una nueva fila. Los ingredientes ya añadidos se muestran como tarjetas editables que pueden eliminarse individualmente.

Pasos de preparación

Una sección dinámica similar a la de ingredientes pero para los pasos. Cada paso se añade con su texto descriptivo. Los pasos se muestran numerados. El botón 'Publish' en la parte inferior envía todos los datos al backend, creando la receta.

8.8 ProfileView — Perfil propio

La pantalla de perfil propio es el espacio personal del usuario dentro de PlateUp. Se organiza en las siguientes secciones:

Tarjeta de usuario

Una tarjeta en la parte superior muestra el avatar en circular de 80x80px con un ring naranja suave, el nombre para mostrar, el username con @, y la biografía del usuario. Debajo, tres métricas en tarjetas redondeadas de fondo naranja y gris: número de recetas publicadas (en naranja, más destacado), número de seguidores (botón interactivo que abre la lista de seguidores), y número de seguidos (botón interactivo que abre la lista de seguidos).

Tabs de contenido

Dos botones tipo tab alternan entre las dos secciones de contenido del perfil: 'My Recipe Book' (mis recetas) y 'Achievements' (mis logros). El tab activo se destaca con el color principal naranja y texto blanco.

My Recipe Book

Muestra todas las recetas publicadas por el usuario en un formato de lista. Cada receta se muestra en un RecipeCard completo con imagen, metadatos y opciones eliminación exclusivas del perfil propio (no visibles en los perfiles de otros usuarios). También se muestra la sección de recetas cocinadas recientemente.

Achievements

Muestra todos los logros obtenidos por el usuario como una lista con el icono del logro, el nombre y los puntos asociados. Los puntos totales acumulados se muestran en un contador destacado en el home del usuario.

Opciones del menú

Un apartado de Settings en la parte inferior muestra un menú con las opciones: 'Edit Profile' (navega a EditProfileView) y 'Log Out' (cierra la sesión limpiando el store de Pinia y redirigiendo al login).

8.9 UserProfileView — Perfil de otros usuarios

La vista del perfil de otro usuario es similar al perfil propio pero con las siguientes diferencias clave: no aparecen las opciones de edición ni eliminación de recetas, y las métricas de seguidores y seguidos son accesibles como en el perfil propio. Al tocar sobre las recetas del usuario, navega al detalle de esa receta.

8.10 EditProfileView — Edición del perfil

La pantalla de edición del perfil permite al usuario actualizar sus datos personales. En la parte superior, una tarjeta destacada de fondo naranja suave muestra el avatar actual con opciones para cambiarlo: 'Change profile photo' abre el selector de archivos del dispositivo, y si hay una imagen nueva seleccionada, el botón 'Remove' permite cancelar la selección. Los cambios de avatar se guardan inmediatamente al seleccionar la imagen.

8.11 ConnectionsView — Seguidores y seguidos

La pantalla de conexiones muestra dos listas de usuarios: seguidores del usuario (las personas que le siguen) y usuarios seguidos (las personas a las que sigue). Un selector en la parte superior alterna entre las dos listas. Cada usuario de la lista se muestra con su avatar, nombre y username. Al tocar un usuario, navega a su perfil. Esta pantalla puede accederse tanto desde el perfil propio como desde el perfil de otro usuario.

8.12 AdminDashboardView — Panel de administración

El panel de administración es una vista exclusiva para usuarios con rol de administrador, accesible mediante la ruta /admin con guard de navegación adminOnly. Muestra estadísticas globales de la plataforma con una interfaz de dashboard:

- KPIs en cuadrícula 2x2: número total de usuarios, número total de recetas, total de likes dados y total de recetas cocinadas totales. Los datos se cargan desde el endpoint del AdminController.
- Gráfico de donut (Doughnut chart) con la distribución porcentual de recetas por categoría, renderizado con Chart.js. Permite visualizar de un vistazo cuáles son las categorías más populares en la plataforma.
- Gráfico de barras con la evolución mensual de nuevos usuarios registrados en los últimos 6 meses. Útil para identificar tendencias de crecimiento de la comunidad.
- Gráfico de barras con el ranking de los usuarios más activos por número de recetas publicadas. Identifica a los creadores de contenido más prolíficos de la plataforma.

9. Mejoras a largo plazo

Aunque PlateUp en su versión actual presenta una plataforma funcional, completa y bien estructurada, existen múltiples líneas de mejora y expansión que se contemplan para versiones futuras del proyecto. Estas mejoras se han priorizado en función de su impacto potencial en la experiencia del usuario y la viabilidad técnica:

9.1 Sistema de notificaciones en tiempo real

La implementación de un sistema de notificaciones push permitiría informar a los usuarios en tiempo real cuando alguien da like a su receta, cuando alguien les sigue, cuando alguien comenta en sus publicaciones o cuando hay un nuevo reto disponible. Técnicamente, esto requeriría la implementación de WebSockets en el backend (con Spring WebSocket o Server-Sent Events) y la integración con el sistema de notificaciones nativo de Android en la versión móvil mediante los plugins de Capacitor.

9.2 Recomendaciones personalizadas con IA

Un sistema de recomendación inteligente analizaría el historial de interacciones de cada usuario (recetas que ha cocinado, categorías preferidas, usuarios que sigue, recetas que ha dado like) para sugerir contenido relevante en el feed y en la sección Explore. Esto podría implementarse con algoritmos de filtrado colaborativo o con modelos de machine learning. La tabla CookedRecipes ya almacena los datos necesarios para alimentar este sistema.

9.3 Integración con compra de ingredientes

Permitir al usuario añadir directamente los ingredientes de una receta a la cesta de la compra de supermercados online como Mercadona, Carrefour o Amazon Fresh. Esta integración generaría ingresos por comisión de afiliado y aportaría un valor añadido significativo a la plataforma. Técnicamente requeriría integraciones con las APIs de los supermercados o con agregadores de precios de alimentación.

9.4 Versión iOS con Capacitor

Dado que el proyecto ya utiliza Capacitor para la versión Android, la extensión a iOS sería relativamente sencilla en términos de código, ya que el mismo proyecto Capacitor puede compilarse para ambas plataformas. Requeriría una cuenta de desarrollador de Apple (Apple Developer Program, 99\$/año), un Mac para compilar el proyecto con Xcode, y pruebas exhaustivas en dispositivos iOS para asegurar la compatibilidad.

9.5 Mensajería directa entre usuarios

Un sistema de chat privado entre usuarios permitiría intercambiar recetas, fotos de platos y consejos de cocina de forma directa. Requeriría WebSockets para mensajería en tiempo real, un modelo de datos adicional (Messages, Conversations) y una vista de chat completa en el frontend.

9.6 Mejoras en el sistema de gamificación

Ampliar el sistema de gamificación con temporadas (seasons) de desafíos con clasificaciones globales, eventos especiales relacionados con festividades gastronómicas (Semana Santa, Navidad, etc.), rankings semanales de los usuarios más activos, y sistema de niveles de experiencia para el perfil de usuario.

9.7 Internacionalización (i18n)

Implementar soporte para múltiples idiomas en la interfaz mediante la librería vue-i18n para el frontend y configuraciones de locale en el backend. La prioridad sería el español, inglés y italiano, con posibilidad de ampliar a otros idiomas europeos.

9.8 Almacenamiento de imágenes en CDN

Migrar el almacenamiento de imágenes del servidor local de Railway a un servicio CDN especializado como Cloudinary, AWS S3 + CloudFront. Esto mejoraría significativamente los tiempos de carga de imágenes gracias a la distribución geográfica del contenido, reduciría la carga del servidor principal y eliminaría la pérdida de imágenes cuando el servidor de Railway se reinicia (en el plan gratuito el almacenamiento local es efímero).

9.9 Sistema de reportes y moderación

Implementar el sistema de reportes de contenido inapropiado cuya tabla (Reports) ya está definida en la base de datos. Los usuarios podrían reportar recetas o comentarios que incumplan las normas de la comunidad. El administrador tendría acceso a una cola de reportes pendientes de revisión desde el panel de administración, con opciones para aprobar, rechazar o eliminar el contenido reportado. Esta tarea se podría automatizar con un agente de IA que decida si eliminar o no una publicación.

9.10 Perfiles públicos y privados con sistema de solicitudes de seguimiento

Implementar configuración de privacidad en perfiles: públicos (visible para todos, recetas en ExploreView) y privados (requiere solicitud de seguimiento aceptada para ver contenido). Incluir notificaciones de solicitudes y gestor de seguimientos en el perfil. Requiere tabla de solicitudes de seguimiento y filtrado en ExploreView.

10. Conclusión

El desarrollo de PlateUp ha supuesto un ejercicio completo e integrador de prácticamente todos los conocimientos adquiridos durante los dos años del ciclo de Desarrollo de Aplicaciones Multiplataforma. Desde el diseño del modelo de datos hasta el despliegue en producción, pasando por el desarrollo del backend, la implementación de la seguridad, el frontend reactivo y la distribución Android, cada componente del proyecto ha requerido aplicar de forma práctica conceptos trabajados en los distintos módulos del ciclo.

El resultado es una aplicación funcional, desplegada en producción y accesible desde cualquier dispositivo, que resuelve un problema real: la inexistencia de una plataforma específica de cocina con la componente social que ofrecen las redes generalistas. PlateUp demuestra que es posible construir un producto tecnológico completo y profesional con las herramientas aprendidas durante el ciclo formativo.

Desde el punto de vista técnico, las decisiones de arquitectura han demostrado ser acertadas. La separación entre frontend y backend ha facilitado el desarrollo paralelo y la prueba independiente de cada componente. La autenticación stateless con JWT ha resultado especialmente conveniente para el despliegue distribuido. El uso de Spring Data JPA ha reducido drásticamente el código repetitivo en la capa de persistencia. Y el framework Vue.js ha demostrado ser una excelente elección por su curva de aprendizaje suave y su reactividad intuitiva.

El proceso de despliegue con Railway y Railway ha sido una experiencia muy valiosa, ya que ha permitido trabajar con herramientas de DevOps reales: contenedores Docker, variables de entorno para la configuración de producción, bases de datos gestionadas en la nube con SSL y pipelines de despliegue automático desde GitHub. Estas son las mismas herramientas y procesos que se utilizan en el desarrollo profesional de software.

Desde el punto de vista personal, el desarrollo de PlateUp ha confirmado que la metodología incremental es la más adecuada para proyectos de esta envergadura: mantener siempre una versión funcional, avanzar en pequeños pasos bien definidos y usar el control de versiones de forma disciplinada son hábitos que marcan la diferencia entre un proyecto que llega a buen puerto y uno que se bloquea antes de estar terminado.

En definitiva, PlateUp es mucho más que un proyecto de fin de ciclo: es una demostración real de competencia técnica, capacidad de planificación y visión de producto. Un proyecto que, con los recursos adecuados, podría evolucionar hacia un producto comercial viable.

11. Anexos

Anexo I. Estructura completa del repositorio

El repositorio de PlateUp en GitHub (<https://github.com/XabierLDGA/PlateUp.git>) tiene la siguiente estructura de directorios:

Directorio / Archivo	Contenido
plateup/	Proyecto backend Java Spring Boot. Contiene pom.xml, Dockerfile, Dockerfile de producción y el directorio src/main/java con todos los paquetes del backend.
plateup/src/main/java/es/ieslavereda/plateup/	Paquete raíz del backend con los subpaquetes: config, controller, dto, exception, model, repository, security, service.
plateup/src/main/resources/	application.properties con la configuración del servidor (datasource, JWT, uploads, HikariCP).
Vue/plateup-frontend/	Proyecto frontend Vue.js 3 con Vite. Contiene package.json, vite.config.js, capacitor.config.json, nginx.conf y Dockerfile.
Vue/plateup-frontend/src/	Código fuente del frontend: views/, components/, router/, stores/, services/, utils/ y assets/.
Vue/plateup-frontend/android/	Proyecto Android generado por Capacitor con AndroidManifest.xml, build.gradle y carpetas de recursos de la app.
db/	Scripts SQL del proyecto: Crear_BaseDatos_Prueba.sql (DDL completo), Inserts_básicos.sql (datos de prueba), Pruebas.sql (script combinado), Limpiar_tablas.sql, Selects_Prueba.sql y Pruebas_aiven.sql.
docker-compose.yml	Orquestación completa de los tres contenedores (MySQL, backend, frontend) para desarrollo local.
railway.toml	Configuración de despliegue en Railway: dos servicios (plateup-backend como Docker, plateup-frontend como Static Site) con sus variables de entorno.
assets/	Recursos gráficos del proyecto: LogoApp.png, LogoPNG.png, LogoPDF.pdf y splash.png para la app Android.
docs/	Documentación del proyecto: PlateUp.pdf, Propuesta inicial, estructura de la memoria, notas de diseño y diagramas entidad-relación en formato PNG y SVG.

Anexo II. Endpoints completos de la API REST

A continuación se listan todos los endpoints de la API REST de PlateUp, con el método HTTP, la ruta y una descripción breve:

Endpoint	Descripción
POST /api/users/register	Registro de nuevo usuario. Body: { username, email, password }. Devuelve 201 con AuthResponse (token JWT + datos usuario).
POST /api/users/login	Inicio de sesión. Body: { username, password }. Devuelve 200 con AuthResponse.
GET /api/users/{id}	Obtener perfil de usuario por ID. Público.
PUT /api/users/{id}	Actualizar datos de perfil. Requiere autenticación y ser el propietario.
GET /api/users/search?q={term}	Buscar usuarios por username o displayName. Público.
GET /api/recipes	Listar todas las recetas ordenadas por fecha. Público.
POST /api/recipes	Crear nueva receta. Requiere autenticación. Body con todos los campos de la receta.
GET /api/recipes/{id}	Obtener detalle completo de una receta. Público.
PUT /api/recipes/{id}	Actualizar receta. Requiere autenticación y ser el propietario.
DELETE /api/recipes/{id}	Eliminar receta. Requiere autenticación y ser el propietario.
GET /api/recipes/user/{userId}	Listar recetas de un usuario. Público.
GET /api/recipe-steps/recipe/{recipeId}	Listar pasos de una receta. Público.
POST /api/recipe-steps	Crear paso de receta. Requiere autenticación.
PUT /api/recipe-steps/{id}	Actualizar paso. Requiere autenticación.
DELETE /api/recipe-steps/{id}	Eliminar paso. Requiere autenticación.
GET /api/recipe-ingredients/recipe/{recipeId}	Listar ingredientes de una receta. Público.
POST /api/recipe-ingredients	Añadir ingrediente a receta. Requiere autenticación.
DELETE /api/recipe-ingredients/{id}	Eliminar ingrediente de receta. Requiere autenticación.
POST /api/likes	Dar like a una receta. Body: { recipeId }. Requiere autenticación.

DELETE /api/likes	Quitar like. Body: { recipeld }. Requiere autenticación.
GET /api/likes/recipe/{recipeld}/count	Contar likes de una receta. Público.
GET /api/likes/user/{userId}/check/{recipeld}	Verificar si un usuario ha dado like a una receta. Público.
POST /api/follows	Seguir usuario. Body: { followedId }. Requiere autenticación.
DELETE /api/follows	Dejar de seguir. Body: { followedId }. Requiere autenticación.
GET /api/follows/{userId}/followers	Listar seguidores de un usuario. Público.
GET /api/follows/{userId}/following	Listar usuarios seguidos. Público.
GET /api/follows/check	Verificar si el usuario actual sigue a otro. Requiere autenticación.
GET /api/achievements	Listar todos los logros del sistema. Público.
GET /api/user-achievements/user/{userId}	Listar logros de un usuario. Público.
GET /api/challenges	Listar todos los retos del sistema. Público.
GET /api/collections/user/{userId}	Listar colecciones de un usuario. Público.
POST /api/collections	Crear nueva colección. Requiere autenticación.
GET /api/comments/recipe/{recipeld}	Listar comentarios de una receta. Público.
POST /api/cooked-recipes	Registrar receta cocinada. Body: { recipeld, elapsedSeconds }. Requiere autenticación.
GET /api/cooked-recipes/user/{userId}	Listar recetas cocinadas por un usuario. Requiere autenticación.
POST /api/upload	Subir imagen (multipart/form-data). Devuelve la URL pública del archivo. Requiere autenticación.
GET /api/admin/stats	Estadísticas globales de la plataforma. Requiere autenticación de administrador.

Anexo III. Variables de entorno

A continuación se listan todas las variables de entorno utilizadas por el sistema en producción:

Variable	Descripción
SPRING_DATASOURCE_URL	URL JDBC completa de conexión a MySQL en Railway, con parámetros SSL requeridos. Ejemplo: jdbc:mysql://host:port/dbname?useSSL=true
SPRING_DATASOURCE_USERNAME	Nombre de usuario de la base de datos MySQL en Railway.
SPRING_DATASOURCE_PASSWORD	Contraseña del usuario de la base de datos MySQL en Railway.
APP_JWT_SECRET	Clave secreta HMAC-SHA256 para firmar los tokens JWT. Mínimo 32 caracteres. Generada automáticamente por Railway(generateValue: true en railway.toml).
APP_UPLOAD_DIR	Directorio del servidor donde se almacenan las imágenes subidas. Valor en producción: /app/uploads.
SPRING_JPA_HIBERNATE_DDL_AUTO	Política de Hibernate para el esquema de base de datos en producción. Valor: none (no modificar el esquema automáticamente).
VITE_API_URL	URL base de la API REST, usada por el frontend en tiempo de construcción. Ejemplo: https://plateup-back.up.railway.app.

Anexo IV. Dependencias del backend (pom.xml)

El proyecto backend de PlateUp tiene las siguientes dependencias Maven declaradas en el pom.xml:

Dependencia	Versión / Descripción
spring-boot-starter-web	3.5.6 Motor web con Apache Tomcat embebido para servir la API REST.
spring-boot-starter-data-jpa	3.5.6 Integración con Spring Data JPA e Hibernate para el acceso a datos.
spring-boot-starter-security	3.5.6 Framework de seguridad para autenticación y control de acceso.
spring-boot-starter-validation	3.5.6 Validación de datos de entrada con Bean Validation (Jakarta).
jjwt-api	0.12.5 API de JSON Web Tokens para generación y validación de JWT.
jjwt-impl	0.12.5 Implementación de JWT con soporte HMAC-SHA256.
jjwt-jackson	0.12.5 Integración de JWT con Jackson para serialización JSON.
mysql-connector-j	8+ Driver JDBC para conexión con MySQL 8.
lombok	Generación automática de getters, setters, constructores y builders.
springdoc-openapi-starter-webmvc-ui	2.8.0 Generación automática de Swagger UI y documentación OpenAPI 3.

Anexo V. Dependencias del frontend (package.json)

El proyecto frontend tiene las siguientes dependencias principales declaradas en package.json:

Dependencia	Versión / Descripción
vue	3.x Framework JavaScript progresivo para la construcción de interfaces de usuario.
vue-router	4.x Enrutador oficial para Vue.js con soporte para SPA y guards de navegación.
pinia	2.x Librería de gestión de estado oficial para Vue 3.
axios	1.x Cliente HTTP para peticiones a la API REST con soporte para interceptores.
lucide-vue-next	Librería de iconos SVG optimizados para Vue 3.
@tailwindcss/vite	4.x Plugin de Vite para integrar Tailwind CSS v4 en el proceso de construcción.
tailwindcss	4.x Framework de estilos utility-first basado en Lightning CSS.
@vitejs/plugin-vue	Plugin oficial de Vite para soporte de componentes SFC de Vue.
vite	6.x Bundler y servidor de desarrollo ultrarrápido para proyectos web modernos.
@capacitor/core	8.x Core de Capacitor para la integración con plataformas nativas.
@capacitor/android	8.x Plugin de Capacitor para la plataforma Android.

Anexo VI. Configuración de Capacitor (Android)

El archivo capacitor.config.json configura el comportamiento de la aplicación en Android. Los parámetros más relevantes son:

Parámetro	Descripción
appId	com.plateup.app Identificador único del paquete Android para publicación en Play Store.
appName	plateup Nombre de la aplicación que aparece en el dispositivo Android.
webDir	dist Directorio con los archivos compilados de Vue.js que Capacitor empaqueta.
server.url	URL del servidor de desarrollo local para hot-reload durante el desarrollo en Android Studio.
android.backgroundColor	Color de fondo de la splash screen durante el inicio de la aplicación.

Anexo VII. Modelo entidad-relación

El diagrama entidad-relación de PlateUp representa la estructura completa de la base de datos relacional. El modelo muestra todas las entidades del sistema (Users, Recipes, RecipeSteps, Ingredients, RecipeIngredients, Utensils, RecipeUtensils, Follows, Likes, Collections, CollectionRecipes, Achievements, UserAchievements, Challenges, UserChallenges, Comments, CookedRecipes y Media) y sus relaciones mediante claves foráneas.

Las relaciones principales incluyen: usuarios que publican recetas, recetas que contienen múltiples pasos e ingredientes, usuarios que interactúan mediante likes y follows, participación en logros y retos, y organización de recetas en colecciones personalizadas. El diseño normalizado en tercera forma normal garantiza la integridad referencial, evita redundancias en los datos y facilita el mantenimiento y escalabilidad del sistema.

El diagrama completo está disponible en: [Mermaid.ai – PlateUp](#)

Anexo VIII. Identidad visual: paleta de colores y tipografía

Filosofía visual

Las decisiones de diseño visual de PlateUp buscan transmitir calidez, apetito y energía sin perder la limpieza que exige una aplicación mobile-first. La paleta se inspira en los tonos cálidos de la gastronomía —el rojo del pimiento asado, el naranja del sofrito, el crema del pan recién horneado— y se traduce en una identidad visual coherente a lo largo de toda la interfaz.

Paleta de colores

COLOR PRIMARIO

Primary	Primary dark	Primary light	Orange-50
#f45b3f	#e24a2e	#ff8a66	#fff3ef
Botones, activos, likes, nav	Hover de botones	Gradientes, fin de degradado	Fondos de badges y alertas

El rojo-naranja coral #f45b3f es el color de acción de PlateUp. Los tonos rojo-naranja están asociados con el apetito, la energía y la sociabilidad, razón por la que aparecen en marcas del sector alimentario como Coca-Cola, McDonald's o Fanta. En la interfaz aparece en botones de acción, filtros activos, indicadores de progreso, contadores de likes y el ícono de navegación activo. Su variante oscura #e24a2e se reserva para el estado hover, aportando feedback visual inmediato.

GRADIENTE PRINCIPAL

#f45b3f → #ff8a66 linear-gradient(135deg)	Cooking Journey & Login Tarjeta de progreso del home y cabecera del login
---	---

El gradiente diagonal del coral al naranja claro se usa en los bloques más prominentes de la interfaz: la cabecera del login y el panel de progreso Cooking Journey en el home. La progresión hacia un tono más luminoso genera sensación de profundidad y dinamismo sin recurrir a imágenes.

FONDOS Y SUPERFICIES

App background	Card	Gray-100	Gray-50
#fff8f5	#ffffff	#f3f4f6	#f9fafb
Fondo global de la app	Tarjetas y modales	Tags secundarios	Fondos internos de sección

El fondo de la app no es blanco puro sino un blanco cálido (#fff8f5) con mínima presencia del tono primario. Esto crea una atmósfera más acogedora que potencia la apetecibilidad de las fotografías de recetas. Las tarjetas usan blanco puro (#ffffff), creando un contraste sutil que las hace destacar sin sombras agresivas.

ESCALA DE GRISES — TEXTOS

Clase Tailwind	Hex	Uso principal
gray-900	#111827	Títulos y textos principales
gray-800	#1f2937	Subtítulos destacados
gray-700	#374151	Labels de formulario
gray-600	#4b5563	Texto de descripción
gray-500	#6b7280	Metadatos y texto secundario
gray-400	#9ca3af	Hints, placeholders, iconos inactivos

Toda la tipografía de la interfaz usa exclusivamente la escala de grises de Tailwind CSS. No se emplea ningún azul oscuro ni color adicional en textos. El coral #f45b3f aparece puntualmente como color de texto en badges de dificultad y contadores del perfil.

COLORES SEMÁNTICOS

Error bg #fef2f2 Fondo mensajes de error	Error text #dc2626 Texto de error	Info bg #fff7ed Mensajes informativos

Los mensajes de error usan fondo #fef2f2 con texto rojo #dc2626, siguiendo la convención universal. Los mensajes informativos usan naranja-50, manteniéndose dentro de la paleta cálida de la marca.

Tipografía

PlateUp usa Inter como fuente principal, con fallback al stack del sistema operativo (ui-sans-serif, system-ui, -apple-system). Inter es una tipografía sans-serif diseñada específicamente para interfaces digitales de alta legibilidad en pantalla. Su carácter geométrico-humanista aporta modernidad sin resultar fría, equilibrio ideal para una aplicación social de cocina.

ESCALA TIPOGRÁFICA

Uso	Tamaño	Peso	Color
Título de aplicación	30px	700	gray-900
H2 destacado (hero)	24px	700	Blanco / gradiente
H2 de sección	20px	700	gray-900

Título de receta (card)	18px	700	gray-900
Cuerpo y descripciones	14px	400	gray-500
Metadatos y badges	12px	500	#f45b3f / gray-500
Subtítulos y etiquetas	11px	500	gray-400

Se usan exclusivamente peso 400 para cuerpo de texto y peso 700 para títulos, evitando pesos intermedios que generarían ambigüedad jerárquica. El tracking-tight se aplica en los títulos principales para ganar densidad visual en móvil.

Tokens de forma — border-radius

El uso sistemático de esquinas muy redondeadas es uno de los rasgos visuales más distintivos de PlateUp. Comunica modernidad, accesibilidad y suavidad, coherente con el espíritu social de la plataforma.

ESCALA DE REDONDEO

Componente	border-radius
Botones de acción y filtros de categoría	9999px (pill)
Secciones hero y panel Cooking Journey	32–36px
Tarjeta de receta (RecipeCard)	24–28px
Cards secundarias y contenedores de formulario	16px
Inputs y tags compactos	8px

Elevación y separación

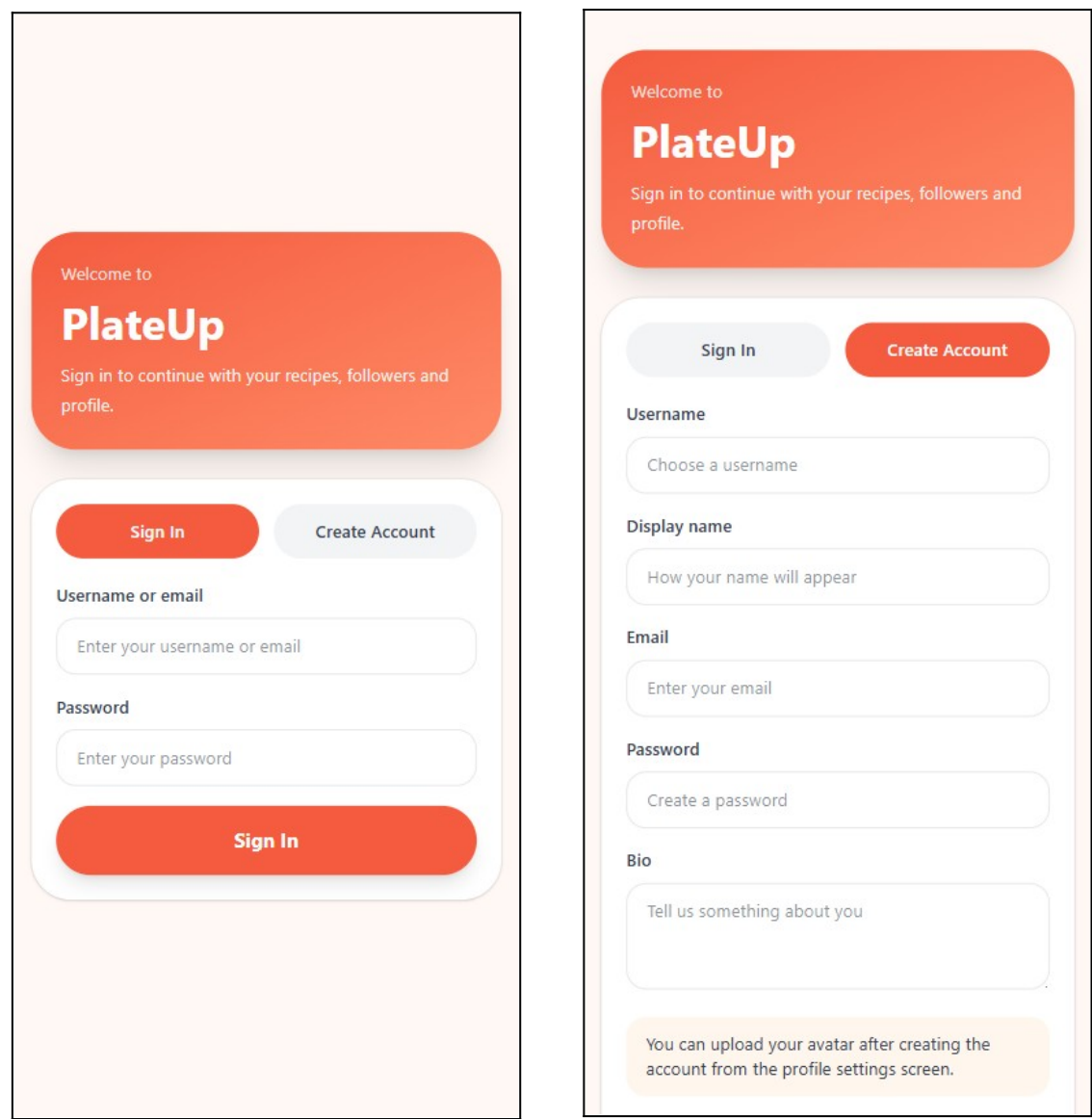
PlateUp prescinde de sombras decorativas y usa ring-1 ring-black/5 —un borde semitransparente de 1px— para separar tarjetas del fondo sin añadir peso visual. Las sombras se reservan para elementos flotantes donde la separación es estructuralmente necesaria.

NIVELES DE SOMBRA

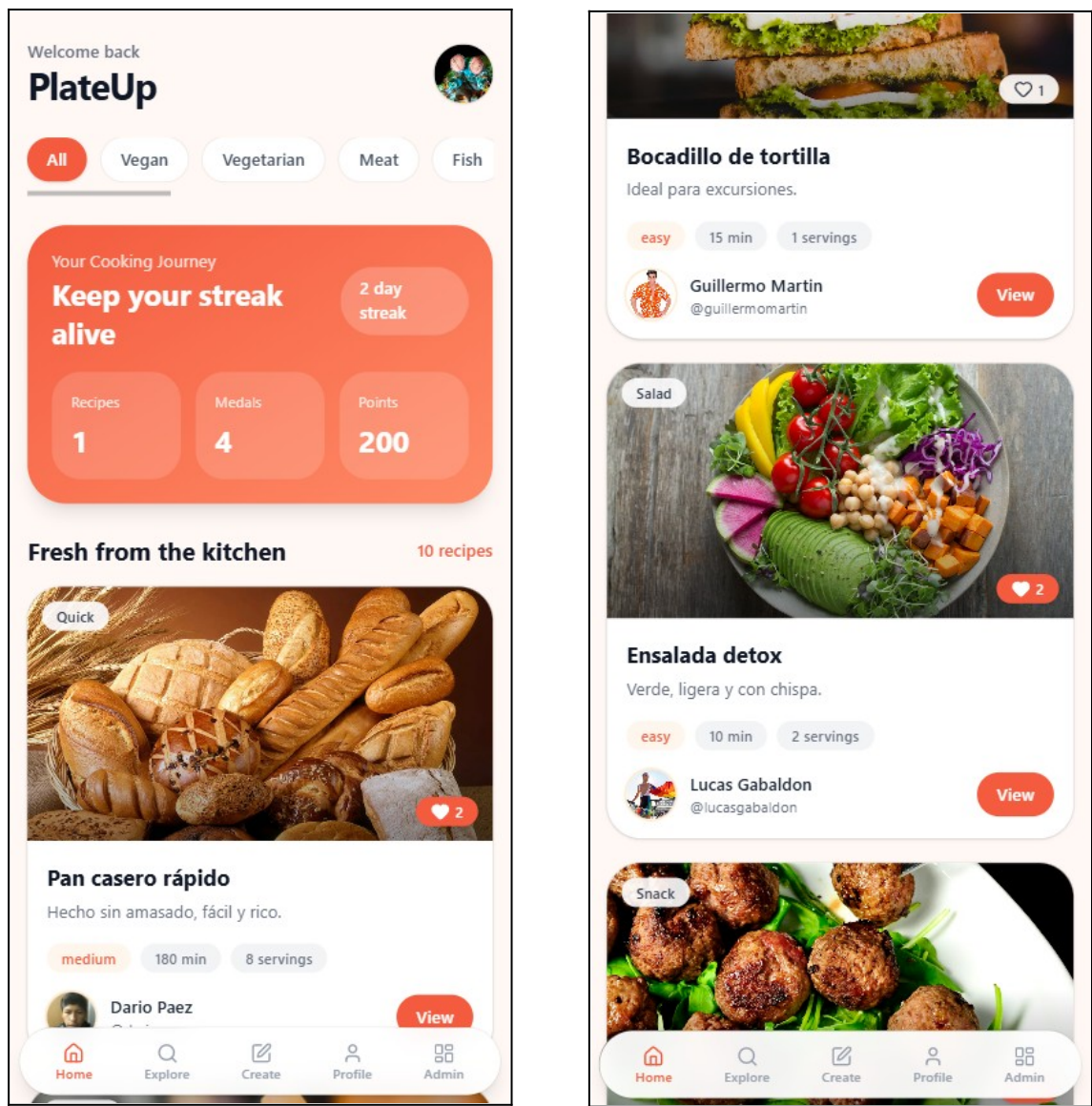
Nivel	Clase Tailwind	Uso
Borde fino	ring-1 ring-black/5	Tarjetas estándar, inputs
Sombra suave	shadow-sm	RecipeCard, listas
Sombra media	shadow-lg	Bloques hero, Cooking Journey
Sombra fuerte	shadow-xl	Login, BottomNav flotante

Anexo IX. Capturas de pantalla de la interfaz

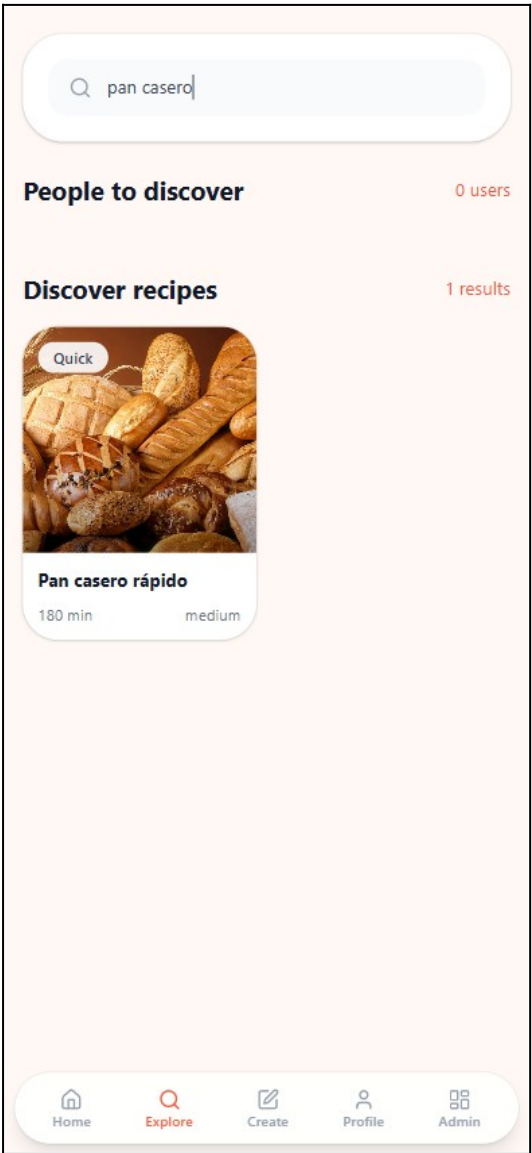
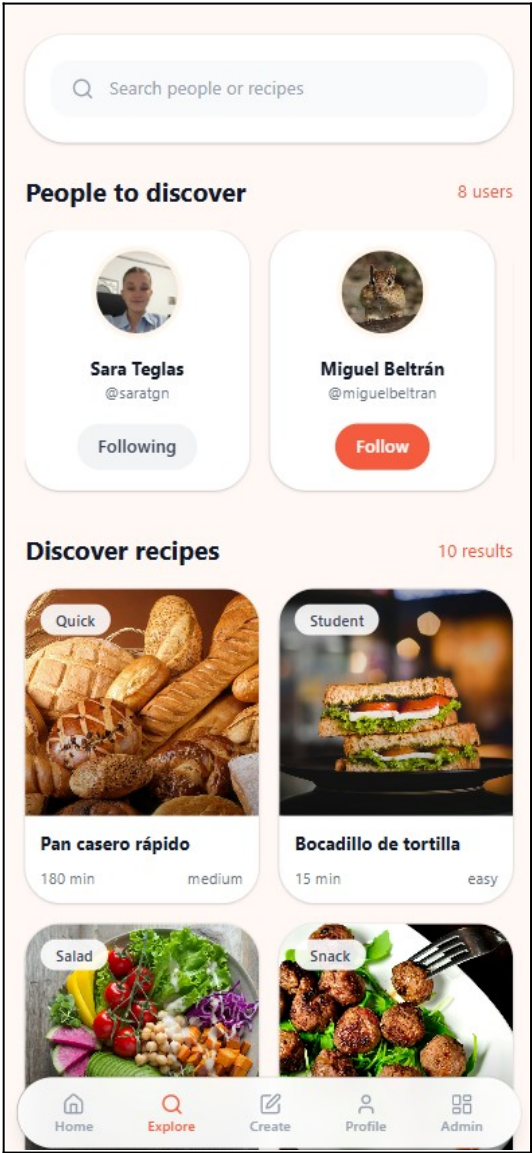
AuthView (Pantalla de autenticación)



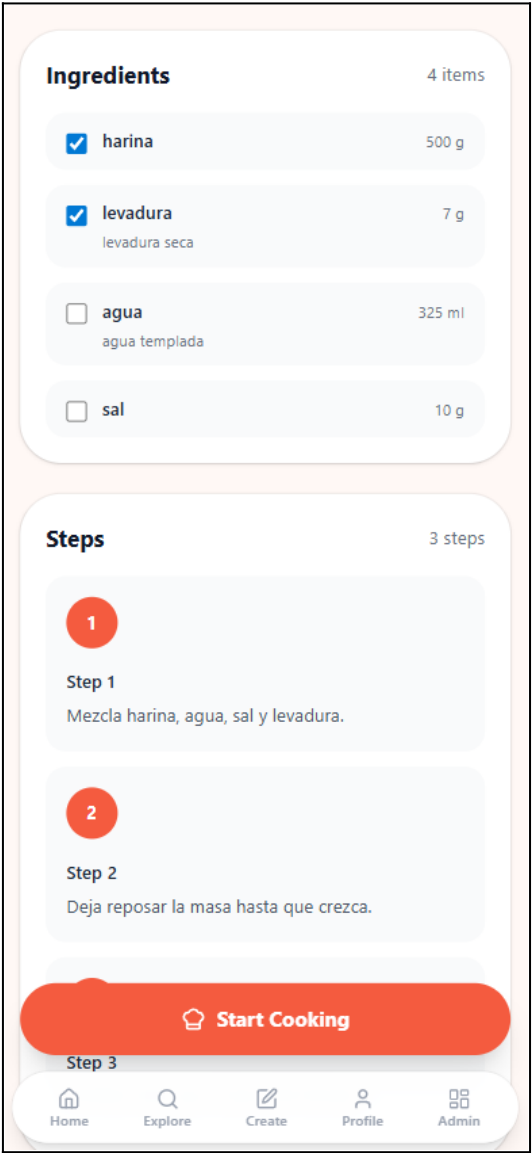
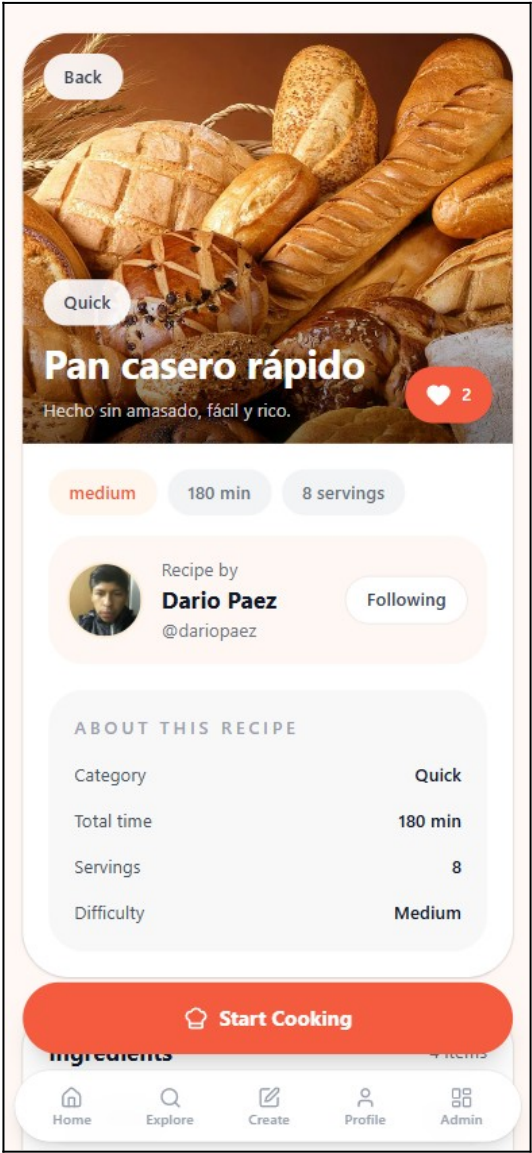
HomeView (Feed principal)



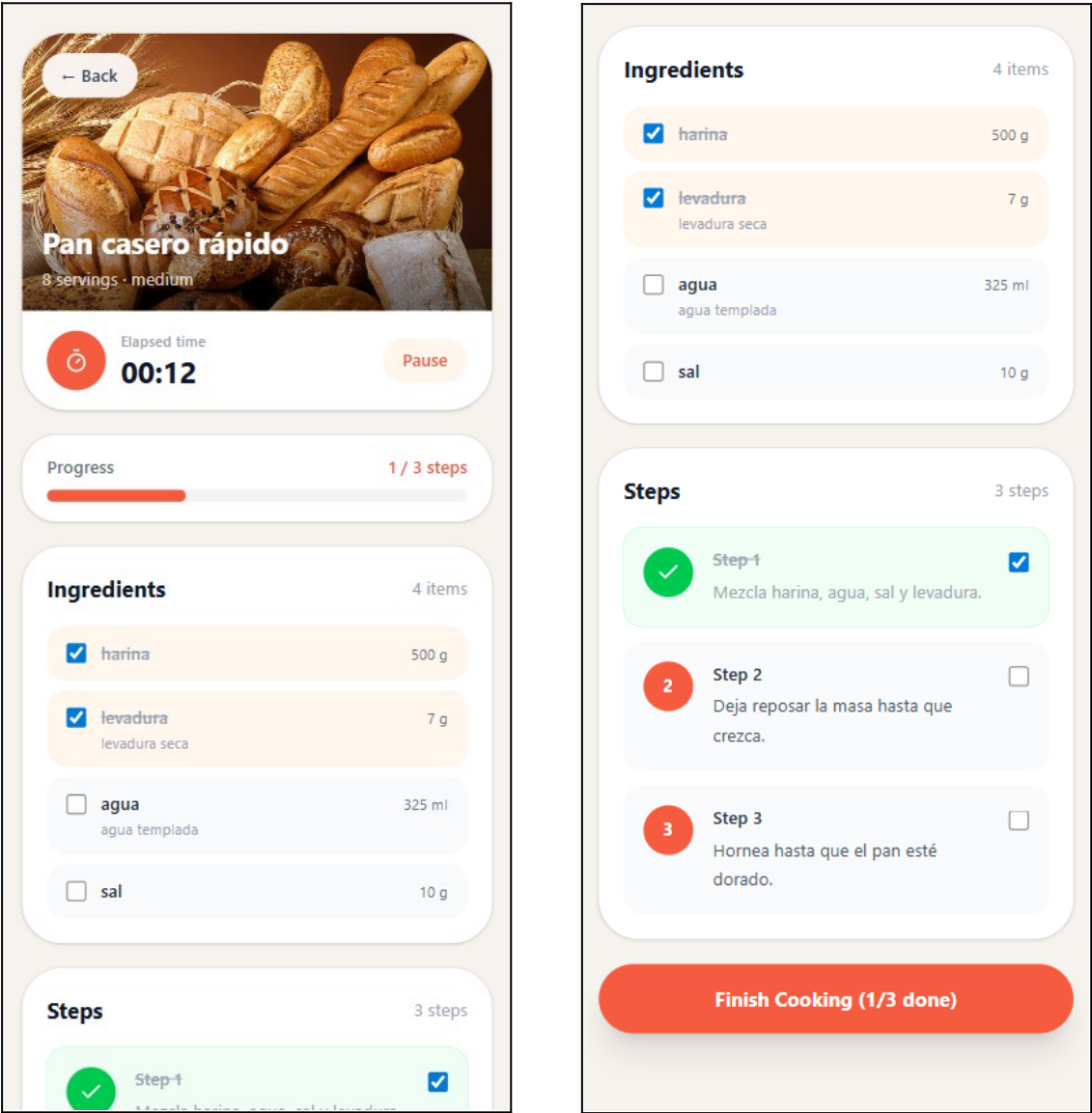
ExploreView (Exploración y búsqueda)



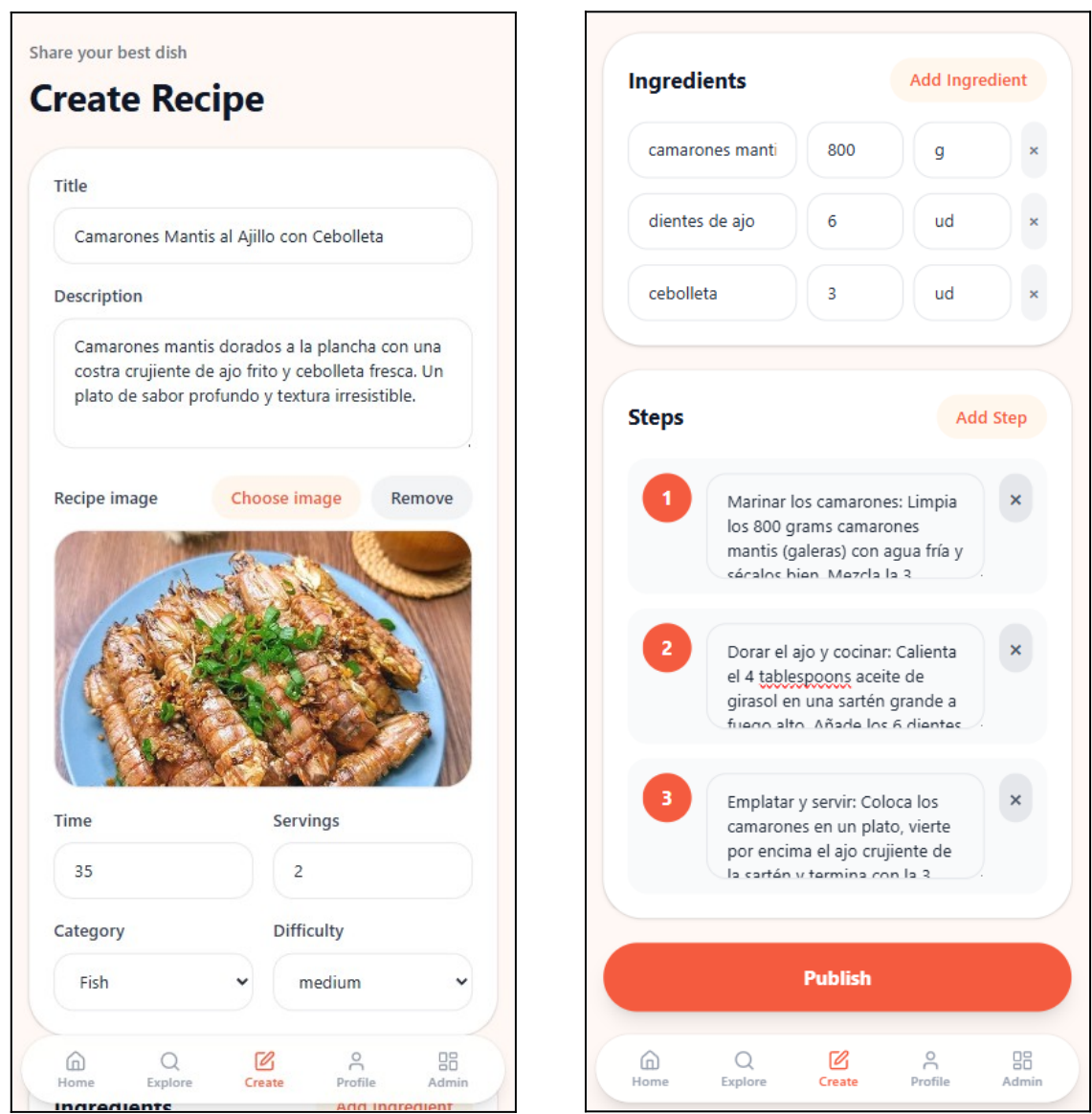
RecipeDetailView (Detalle de receta)



CookingView (Modo cocina guiado)



CreateView (Creación de receta)



ProfileView (Perfil propio)



Camarón Mantis

@camaron

Soy un camarón mantis.

Recipes

1

Followers

4

Following

9

My Recipe Book

Achievements

My Recipe Book



Pasta al pesto

25 min · easy

Delete recipe

Cooked Recipes 2



Home

Explore

Create

Profile

Admin



Camarón Mantis

@camaron

Soy un camarón mantis.

Recipes

1

Followers

4

Following

9

My Recipe Book

Achievements

Achievements



Tu primera receta

50 pts

Publica tu primera receta en PlateUp.

recipes



Primer aplauso

40 pts

Recibe tu primer like en una de tus recetas.

likes



Primera receta cocinada

50 pts

Termina de cocinar tu primera receta.

cooked



Tu primer seguidor

60 pts

Consigue tu primer seguidor.

followers

Home

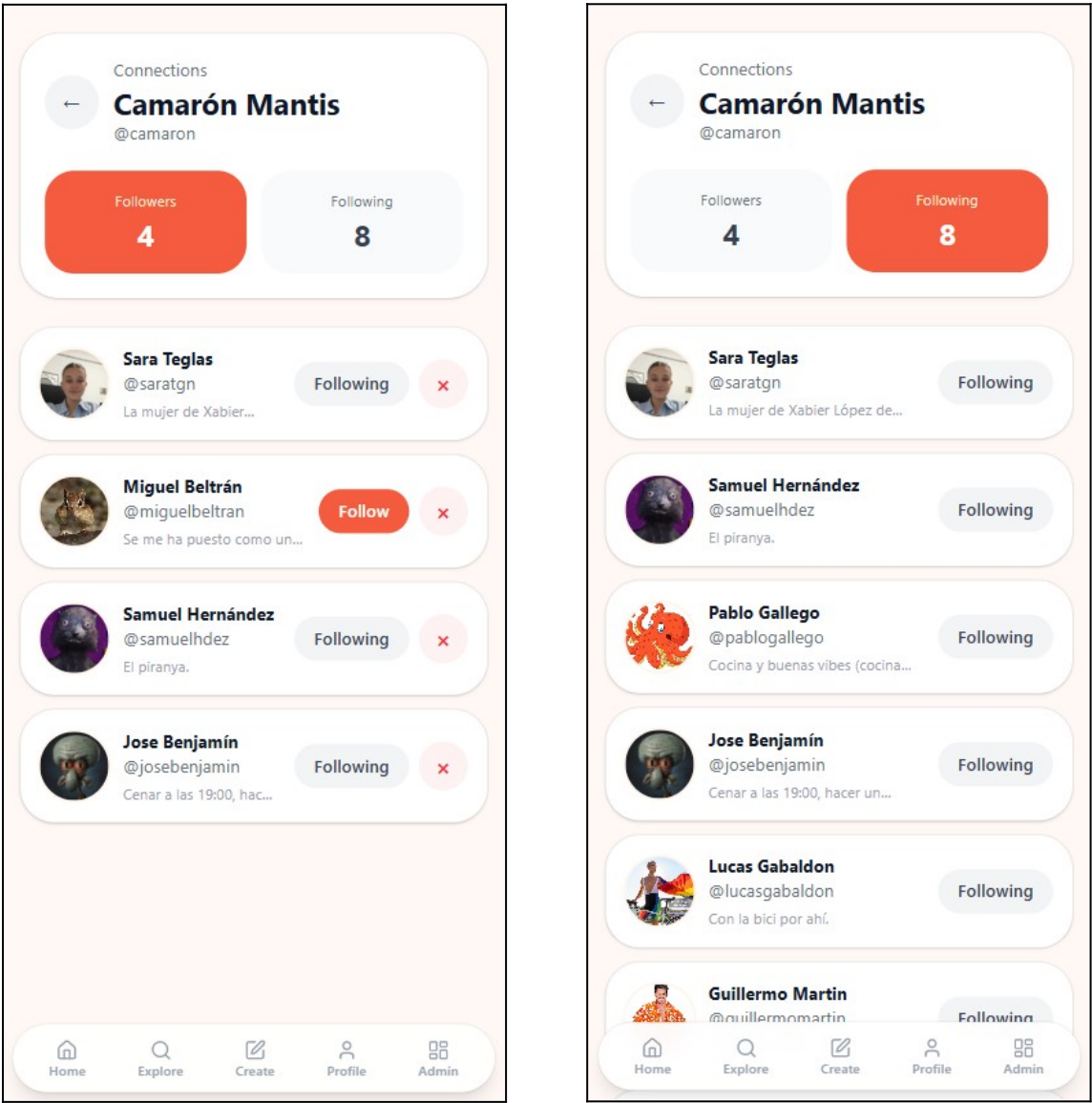
Explore

Create

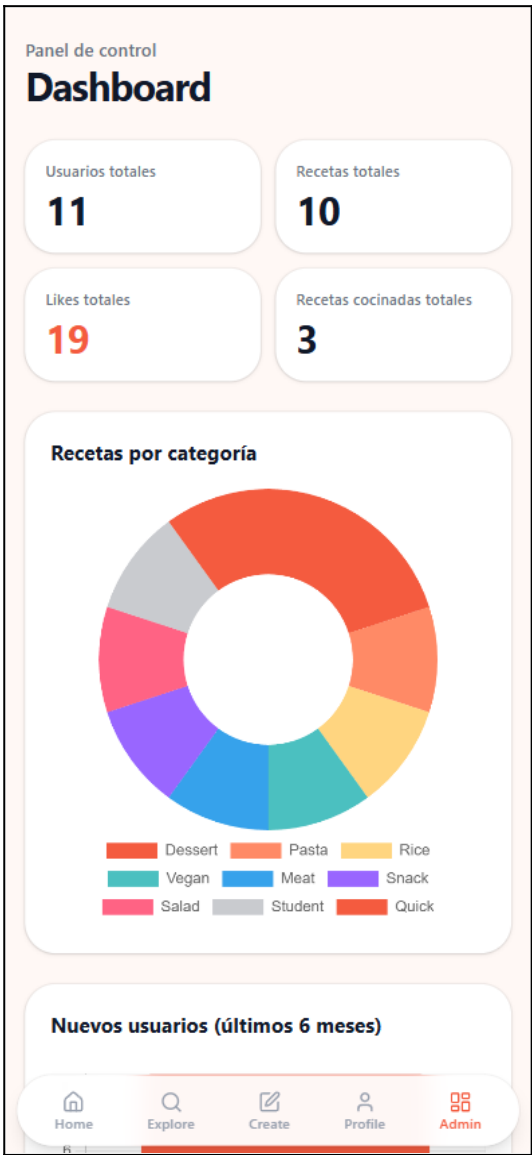
Profile

Admin

ConnectionsView (Panel de followers)



AdminDashboardView (Panel de administración)



Anexo X. Repositorio de GitHub

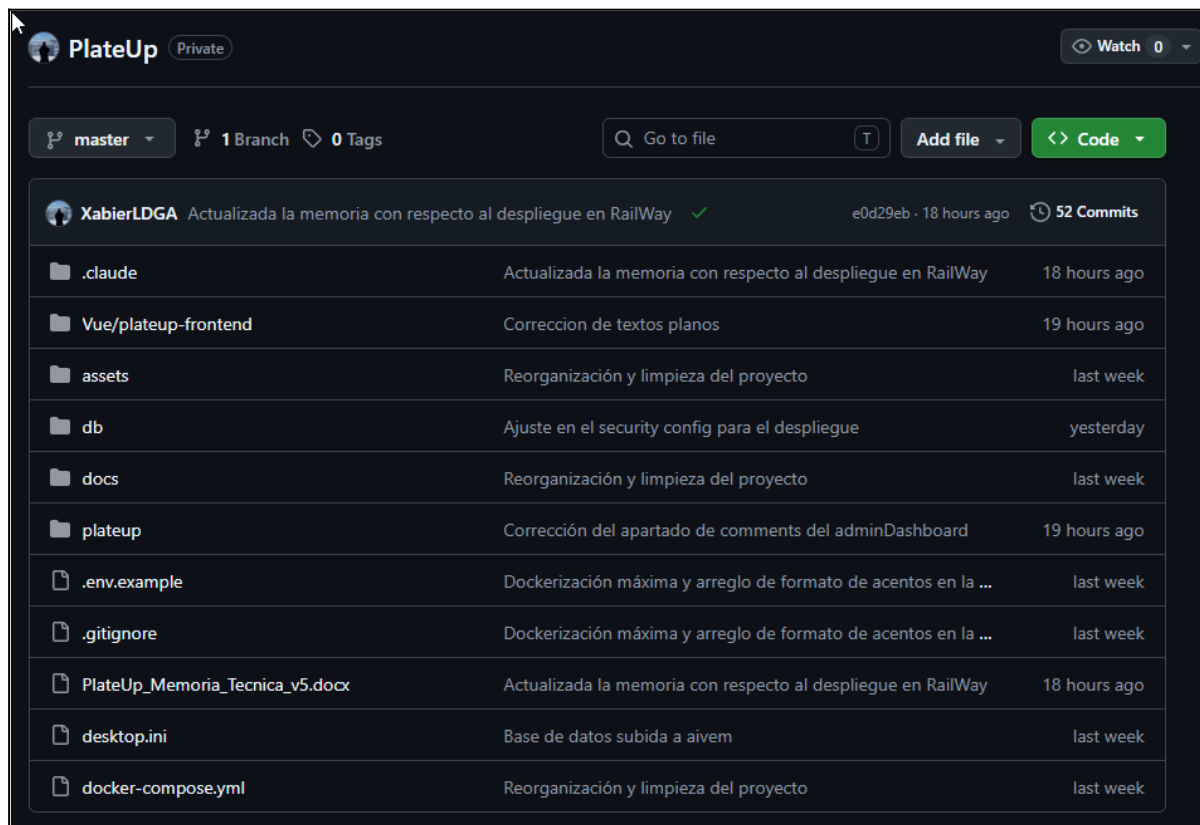
El repositorio de PlateUp está disponible públicamente en GitHub en la siguiente URL: <https://github.com/XabierLDGA/PlateUp.git>. El repositorio centraliza todo el código fuente del proyecto (backend, frontend y configuración de despliegue) y actúa además como punto de integración con Railway para el despliegue continuo automático.

Estructura del repositorio

El repositorio tiene una estructura monorepo con dos proyectos principales en la raíz: la carpeta `plateup/` contiene el backend Spring Boot con su `Dockerfile` y el archivo `pom.xml`; la carpeta `plateup-frontend/` contiene la aplicación Vue.js con su `package.json`, `vite.config.js` y la configuración de Capacitor para Android. En la raíz del repositorio se encuentran los archivos de configuración global: `docker-compose.yml` para el entorno de desarrollo local, `railway.toml` para el despliegue en Railway, y `nginx.conf` para la configuración del servidor web del frontend en producción.

Historial de commits y evolución del proyecto

El historial de commits refleja la metodología iterativa e incremental seguida durante el desarrollo. Los commits se realizaron con frecuencia y mensajes descriptivos que identifican claramente la funcionalidad implementada en cada punto del desarrollo, permitiendo seguir la evolución del proyecto fase a fase. La rama principal `master` contiene el historial completo del desarrollo desde la configuración inicial del proyecto Spring Boot hasta la versión final desplegada en producción.



The screenshot displays the GitHub interface for the `PlateUp` repository, which is marked as `Private`. The repository is currently on the `master` branch, with 1 branch and 0 tags. The commit history shows a recent commit by `XabierLDGA` titled "Actualizada la memoria con respecto al despliegue en RailWay" (e0d29eb, 18 hours ago), which is the 52nd commit. Below the commit history, a list of files and folders is shown, including `.claude`, `Vue/plateup-frontend`, `assets`, `db`, `docs`, `plateup`, `.env.example`, `.gitignore`, `PlateUp_Memoria_Tecnica_v5.docx`, `desktop.ini`, and `docker-compose.yml`. Each item is accompanied by a description of the changes and the time since the last commit.

File/Folder	Description	Time
<code>.claude</code>	Actualizada la memoria con respecto al despliegue en RailWay	18 hours ago
<code>Vue/plateup-frontend</code>	Correccion de textos planos	19 hours ago
<code>assets</code>	Reorganización y limpieza del proyecto	last week
<code>db</code>	Ajuste en el security config para el despliegue	yesterday
<code>docs</code>	Reorganización y limpieza del proyecto	last week
<code>plateup</code>	Corrección del apartado de comments del adminDashboard	19 hours ago
<code>.env.example</code>	Dockerización máxima y arreglo de formato de acentos en la ...	last week
<code>.gitignore</code>	Dockerización máxima y arreglo de formato de acentos en la ...	last week
<code>PlateUp_Memoria_Tecnica_v5.docx</code>	Actualizada la memoria con respecto al despliegue en RailWay	18 hours ago
<code>desktop.ini</code>	Base de datos subida a aivem	last week
<code>docker-compose.yml</code>	Reorganización y limpieza del proyecto	last week

Anexo XI. Configuración del tiempo de expiración de tokens JWT

El tiempo de expiración de los tokens JWT en PlateUp está configurado en 30 días (2.592.000.000 milisegundos). Esta decisión de diseño equilibra seguridad y comodidad de uso: un tiempo demasiado corto obligaría al usuario a iniciar sesión con frecuencia, degradando la experiencia; un tiempo indefinido supondría un riesgo de seguridad si el token cayera en manos equivocadas.

Configuración en application.properties

El tiempo de expiración y la clave secreta del JWT se configuran como propiedades externas en el archivo application.properties, evitando incluir valores sensibles directamente en el código fuente:

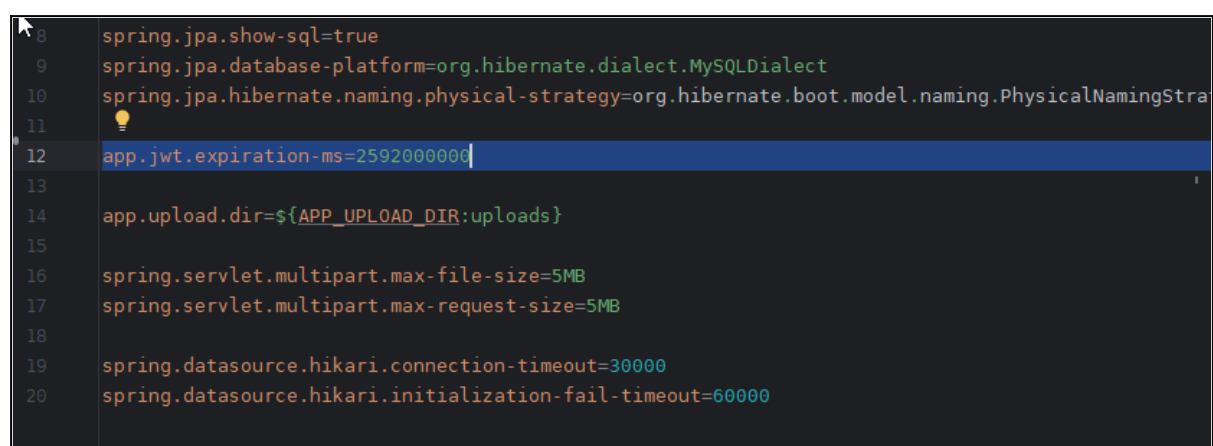
```
app.jwt.secret=${APP_JWT_SECRET}
app.jwt.expiration-ms=2592000000
```

El valor 2592000000 corresponde a 30 días expresados en milisegundos ($30 \times 24 \times 60 \times 60 \times 1000$). La clave secreta se lee de la variable de entorno APP_JWT_SECRET, que en producción es generada automáticamente por Railway con generateValue: true, garantizando que nunca se incluya en el código fuente del repositorio.

Implementación en JwtService

La clase JwtService lee ambas propiedades mediante la anotación @Value de Spring e implementa los métodos de generación y validación del token. El método generateToken() construye el JWT con los claims del usuario (username, userId, email), establece la fecha de emisión y calcula la fecha de expiración sumando el tiempo configurado a la fecha actual. El método isValidToken() verifica simultáneamente que la firma sea correcta y que la fecha de expiración no haya sido superada, rechazando cualquier token manipulado o caducado.

Para modificar el tiempo de expiración en el futuro, basta con cambiar el valor de la propiedad app.jwt.expiration-ms sin necesidad de modificar el código Java. Por ejemplo, para una aplicación bancaria se reduciría a 900000 (15 minutos), mientras que para una aplicación de uso ocasional podría ampliarse a 7776000000 (90 días).

A screenshot of a code editor showing the application.properties file. The file contains various Spring configuration properties. Line 12 is highlighted, showing 'app.jwt.expiration-ms=2592000000'. Other visible properties include database dialect, naming strategy, upload directory, multipart file size, and Hikari connection timeouts.

```
8 spring.jpa.show-sql=true
9 spring.jpa.database-platform=org.hibernate.dialect.MySQLDialect
10 spring.jpa.hibernate.naming.physical-strategy=org.hibernate.boot.model.naming.PhysicalNamingStrat
11
12 app.jwt.expiration-ms=2592000000
13
14 app.upload.dir=${APP_UPLOAD_DIR:uploads}
15
16 spring.servlet.multipart.max-file-size=5MB
17 spring.servlet.multipart.max-request-size=5MB
18
19 spring.datasource.hikari.connection-timeout=30000
20 spring.datasource.hikari.initialization-fail-timeout=60000
```

Anexo XII. Pruebas con Swagger UI

Swagger UI fue la herramienta principal utilizada para probar los endpoints de la API REST de PlateUp durante el desarrollo. Al estar integrada directamente en el proyecto mediante springdoc-openapi, permite probar cualquier endpoint desde el navegador sin necesidad de herramientas externas, utilizando la misma documentación generada automáticamente a partir de las anotaciones de los controladores.

Organización de la colección

Swagger UI organiza automáticamente todos los endpoints por controlador REST: Auth (registro y login), Users, Recipes, RecipeSteps, RecipeIngredients, Likes, Follows, Achievements, UserAchievements, Challenges, UserChallenges, Collections, Comments, CookedRecipes, Upload y Admin. Cada grupo muestra los métodos HTTP disponibles con su esquema de petición y respuesta, permitiendo ejecutar pruebas directamente desde el navegador.

Casos de prueba principales

Para cada endpoint se verificaron los siguientes casos: respuesta correcta con datos válidos (código 200 o 201 según corresponda), comportamiento ante datos inválidos o incompletos (código 400 Bad Request), comportamiento sin token JWT en endpoints protegidos (código 401 Unauthorized), comportamiento con token JWT inválido o expirado (código 401), e intento de acceder a recursos que no existen (código 404 Not Found). Los endpoints de administración se probaron adicionalmente con usuarios sin rol de administrador para verificar que el acceso se denegaba correctamente.

Servers	
http://localhost:8080 - Generated server url	
utensil-controller	▼
user-controller	▼
user-challenge-controller	▼
user-achievement-controller	▼
recipe-step-controller	▼
recipe-controller	▼
ingredient-controller	▼
follow-controller	▼
comment-controller	▼
collection-controller	▼
collection-recipe-controller	▼
challenge-controller	▼
achievement-controller	▼
upload-controller	▼
recipe-ingredient-controller	▼
like-controller	▼
cooked-recipe-controller	▼
admin-controller	▼

Resultados de las pruebas

Todos los endpoints de la API REST superaron las pruebas con el resultado esperado en ambos entornos: local (localhost:8080) y producción (Railway). Los 18 controladores respondieron correctamente con los códigos HTTP apropiados, los cuerpos de respuesta JSON bien formados y el comportamiento correcto de la autenticación JWT.

POST /api/users/register

Parameters

No parameters

Request body required

application/json

```
{
  "username": "testuser",
  "email": "testuser@plateup.com",
  "password": "Password123!",
  "displayName": "Usuario de Prueba",
  "bio": "Me encanta cocinar recetas mediterráneas.",
  "avatarUrl": "",
  "visibilityDefault": "public"
}
```

Execute

Responses

Responses

Curl

```
curl -X 'POST' \
'http://localhost:8080/api/users/register' \
-H 'accept: */*' \
-H 'Content-Type: application/json' \
-d '{
  "username": "testuser",
  "email": "testuser@plateup.com",
  "password": "Password123!",
  "displayName": "Usuario de Prueba",
  "bio": "Me encanta cocinar recetas mediterráneas.",
  "avatarUrl": "",
  "visibilityDefault": "public"
}'
```

Request URL

```
http://localhost:8080/api/users/register
```

Server response

Code

Details

201

Undocumented

Response body

```
{
  "token": "eyJhbGciOiJIUzUxMiJ9.eyJzdWIiOiJoZXN0dXNiIiwiaWF0IjoxNzY3ZCJ2ZC6MTIsImVtYWkiOiJoidGVzdHVzZXJAcGxhdGV1cC5jb20iLCJyYbZkl1Ijo1VWVnFUIiIsImhhdCI6MTc3OTM1ODczO0wiLCJ0eWtIjozNzgxOTUwNzY4FQ.DFTvngU473DDQqncJg_F2vNsZ1A1P0scCpkuk-BstFP0_YbsNdIG4Dv0JucCuaVzMcmvoJTrPKUAjKwq01Yw",
  "user": {
    "id": 12,
    "username": "testuser",
    "email": "testuser@plateup.com",
    "displayName": "Usuario de Prueba",
    "bio": "Me encanta cocinar recetas mediterráneas.",
    "avatarUrl": "",
    "visibilityDefault": "public",
    "createdAt": "2026-05-21T12:18:58.7059023",
    "updatedAt": "2026-05-21T12:18:58.7059023",
    "streakCount": 0,
    "lastActiveDate": null,
    "role": "USER"
  }
}
```

Download

Response headers